TruthLens AI: An Explainable Multilingual Multimodal System for Fake News and AI-Generated Image Detection

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted by

Batch B03

K. Srivalli (23481A1299)

I. Bhargavi (23481A1289)

I. Teja Sai (23481A1288)

K. Lakshmi (23481A12A3)

Under the guidance of

Sri Y. K. Viswanadham, M.Tech., Ph.D.

Associate Professor, Dept. of IT

DEPARTMENT OF INFORMATION TECHNOLOGY

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

Seshadri Rao Knowledge Village, Gudlavalleru – 521356, Andhra Pradesh

2026 – 2027

DEPARTMENT OF INFORMATION TECHNOLOGY

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

CERTIFICATE

This is to certify that the project report entitled **"TruthLens AI: An Explainable Multilingual Multimodal System for Fake News and AI-Generated Image Detection"** is a bonafide record of work carried out by **K. Srivalli (23481A1299)**, **I. Bhargavi (23481A1289)**, **I. Teja Sai (23481A1288)**, **K. Lakshmi (23481A12A3)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology from Seshadri Rao Gudlavalleru Engineering College, during the academic year 2026 – 2027.

Project Guide
Sri Y. K. Viswanadham, M.Tech., Ph.D.
Associate Professor, Dept. of IT

Head of the Department
Professor & Head, Dept. of IT

External Examiner

DECLARATION

We hereby declare that the project report entitled **"TruthLens AI: An Explainable Multilingual Multimodal System for Fake News and AI-Generated Image Detection"** submitted to the Department of Information Technology, Seshadri Rao Gudlavalleru Engineering College, is a record of original work done by us under the guidance of Sri Y. K. Viswanadham, M.Tech., Ph.D., Associate Professor, Dept. of IT. The results in this report have not been submitted to any other university or institute for the award of any degree or diploma.

K. Srivalli (23481A1299)

I. Bhargavi (23481A1289)

I. Teja Sai (23481A1288)

K. Lakshmi (23481A12A3)

ACKNOWLEDGEMENT

We thank our guide, Sri Y. K. Viswanadham, M.Tech., Ph.D., Associate Professor, Department of Information Technology, for the guidance, encouragement and time given to us at every stage of this project.

We thank the Head of the Department of Information Technology for providing the facilities and support needed to complete this work, and the project review committee for their comments at each review.

We thank the Principal and the Management of Seshadri Rao Gudlavalleru Engineering College for the environment and resources that made this project possible.

We are grateful to the teaching and non-teaching staff of the Department of Information Technology for their help during the course of our study.

We thank the creators of the open datasets and models this project depends on, in particular the authors of WELFake, CIFAKE, FEVER and Climate-FEVER, the AI4Bharat group for IndicBERT and IndicXNLI, and the developers of Hugging Face Transformers, Kaggle and Google Colab, whose free GPUs made the training possible.

Finally, we thank our parents and friends for their patience and encouragement.

ABSTRACT

False news spreads faster than corrections, and readers in Andhra Pradesh and Telangana meet it in both English and Telugu, often next to a convincing AI-generated image. Most automatic detectors handle only English text, give one label for a whole article and do not explain it. This project presents TruthLens AI, a web application that classifies English and Telugu news, detects AI-generated images, explains every verdict, and checks the individual claims in an article against evidence.

English news is classified by DeBERTa-v3-large and Telugu news by IndicBERT v2, both fine-tuned on a free T4 GPU. The English model was trained on the WELFake dataset after its reversed labels were corrected and publisher names were removed; because the public Telugu dataset could be separated at 99.8% by word counts alone, a Telugu set was built by translating WELFake with NLLB-200. Images are classified by a linear head on frozen CLIP ViT-L/14 features. LIME marks the words behind each text verdict and Grad-CAM shows the image regions behind each image verdict. For claim verification, the article is split into check-worthy claims, evidence is retrieved with BGE-M3 embeddings and FAISS, and a natural language inference model labels each claim Supported, Contradicted or Unknown, with contradiction tested first.

The English classifier reached 99.7% accuracy on 12,727 test articles, against 98.8% for BERT-base, and the Telugu classifier reached 93.1%, against 87.1% for a TF-IDF baseline. The image detector reached 93.7% on CIFAKE. On FEVER, claim verification reached 84.6% accuracy and 0.847 macro-F1, while a similarity-only baseline reached 40.0% and found no contradictions; on the harder, real-world Climate-FEVER set it reached 50.9%, against 34.0% for the baseline. The system is built with FastAPI and React, supports batch checking and history, and installs and starts with one command.

Keywords: fake news detection, Telugu, DeBERTa, IndicBERT, CLIP, explainable AI, claim verification, natural language inference.

TABLE OF CONTENTS

CERTIFICATE iii
DECLARATION iv
ACKNOWLEDGEMENT v
ABSTRACT vi
TABLE OF CONTENTS vii
LIST OF FIGURES xii
LIST OF TABLES xiv
LIST OF ABBREVIATIONS xv
1. INTRODUCTION 2
1.1 Overview 2
1.2 Problem Statement 2
1.3 Motivation 3
1.4 Objectives 3
1.5 Scope of the Project 4
1.6 Methodology in Brief 5
1.7 Organisation of the Report 5
2. LITERATURE SURVEY 6
2.1 Fake News Detection from Text 6
2.1.1 Feature-based methods 6
2.1.2 Benchmark datasets 6
2.1.3 Deep learning and transformers 6
2.2 Indian-Language Resources 7
2.3 AI-Generated Image Detection 7
2.4 Explainable AI 8
2.5 Automated Fact-Checking 8
2.6 Comparison of Existing Work 9
2.7 Research Gaps 11
3. SYSTEM ANALYSIS 12
3.1 Existing System 12
3.1.1 Disadvantages of the existing system 12
3.2 Proposed System 13
3.2.1 Advantages of the proposed system 13
3.3 Feasibility Study 14
3.3.1 Technical feasibility 14
3.3.2 Economic feasibility 14

3.3.3 Operational feasibility 14
3.4 Requirement Specification 14
3.4.1 Functional requirements 14
3.4.2 Non-functional requirements 15
3.4.3 Hardware requirements 16
3.4.4 Software requirements 16
3.5 Software Development Model 16
4. SYSTEM DESIGN 17
4.1 System Architecture 17
4.2 Module Description 19
4.2.1 User interface module 19
4.2.2 Language detection module 19
4.2.3 Text classification module 19
4.2.4 Explanation module 19
4.2.5 Claim verification module 20
4.2.6 Image detection module 20
4.2.7 Batch module 20
4.2.8 History module 20
4.3 UML Diagrams 20
4.3.1 Use case diagram 20
4.3.2 Class diagram 21
4.3.3 Sequence diagram: checking news text 22
4.3.4 Sequence diagram: checking an image 23
4.3.5 Activity diagram: claim verification 23
4.3.6 Activity diagram: overall flow 25
4.3.7 Deployment diagram 25
4.4 Data Flow Diagrams 26
4.5 Database Design 27
4.6 Interface Design 28
4.6.1 REST API 28
4.6.2 Response format 29
4.6.3 User interface design 29
5. METHODOLOGY 30
5.1 Overview 30
5.2 Datasets 31
5.2.1 WELFake 32
5.2.2 Telugu: why the public dataset was rejected 33

5.2.3 Telugu: building a translated set 33
5.2.4 CIFAKE 34
5.3 Preprocessing 34
5.4 Text Classification Models 34
5.4.1 DeBERTa-v3-large for English 34
5.4.2 BERT-base baseline 35
5.4.3 TF-IDF baseline 35
5.4.4 IndicBERT v2 for Telugu 35
5.5 Image Detection Model 35
5.6 Claim Verification Algorithm 36
5.6.1 Step 1: claim extraction 36
5.6.2 Step 2: evidence retrieval 36
5.6.3 Step 3: natural language inference 37
5.6.4 Step 4: decision rule 37
5.6.5 Step 5: aggregation 37
5.7 Explainability Methods 38
5.7.1 LIME for text 38
5.7.2 Grad-CAM for images 38
5.8 Fusion of Classifier and Evidence 39
5.9 Training Setup 39
5.10 Evaluation Metrics 39
6. IMPLEMENTATION 41
6.1 Technologies Used 41
6.2 Project Structure 41
6.3 Backend Implementation 42
6.3.1 Configuration 42
6.3.2 Lazy model loading 42
6.3.3 Language detection 43
6.3.4 Claim decision and fusion 43
6.3.5 Evidence index 43
6.3.6 Image detector and Grad-CAM 43
6.3.7 API and error handling 44
6.4 Frontend Implementation 44
6.4.1 Pages and routing 44
6.4.2 Verdict first, explanation second 44
6.4.3 Marked-up article 45
6.4.4 Theme and accessibility 45

6.5 Training Notebook 45
6.6 One-Command Launcher 46
7. TESTING 47
7.1 Testing Strategy 47
7.2 Unit Testing 47
7.3 Integration Test Cases 48
7.4 System Testing 51
7.5 Performance 52
7.6 Test Summary 53
8. RESULTS AND DISCUSSION 55
8.1 English Text Classification 55
8.2 Telugu Text Classification 57
8.3 AI-Generated Image Detection 59
8.4 Claim Verification 59
8.4.1 FEVER 60
8.4.2 Climate-FEVER 61
8.4.3 Telugu verification 61
8.4.4 Curated set (full pipeline) 62
8.4.5 Choice of threshold 63
8.5 Sample Outputs 64
8.6 Discussion 73
9. CONCLUSION AND FUTURE SCOPE 76
9.1 Conclusion 76
9.2 Limitations 77
9.3 Future Work 77
10. INSTALLATION AND USER GUIDE 79
10.1 Prerequisites 79
10.2 Installing and Starting 79
10.3 Installing the Trained Models 80
10.4 Configuration 80
10.5 Adding Evidence 81
10.6 Using the Application 81
10.6.1 Start page 81
10.6.2 Checking news text 81
10.6.3 Checking an image 82
10.6.4 Verifying claims only 82
10.6.5 Checking many at once 82

10.6.6 History and saving reports 82
10.6.7 Light and dark theme 82
10.7 Troubleshooting 82
REFERENCES 84

LIST OF FIGURES

Figure 4.1: Overall architecture of TruthLens AI 18
Figure 4.2: Use case diagram 21
Figure 4.3: Class diagram of the backend 21
Figure 4.4: Sequence diagram for checking news text 22
Figure 4.5: Sequence diagram for checking an image 23
Figure 4.6: Activity diagram of claim verification 24
Figure 4.7: Activity diagram of the overall check 25
Figure 4.8: Deployment diagram 26
Figure 4.9: Level 0 data flow diagram 26
Figure 4.10: Level 1 data flow diagram 27
Figure 4.11: Entity relationship diagram 27
Figure 5.1: Training pipeline run in the notebook 31
Figure 8.1: Confusion matrix of DeBERTa-v3-large on the English test set 56
Figure 8.2: Training and validation loss of DeBERTa-v3-large 56
Figure 8.3: Confusion matrix of IndicBERT v2 on the Telugu test set 58
Figure 8.4: Training and validation loss of IndicBERT v2 58
Figure 8.5: Confusion matrix of the CLIP image detector on CIFAKE 59
Figure 8.6: Confusion matrix of claim verification on FEVER 60
Figure 8.7: Confusion matrix of claim verification on Climate-FEVER 61
Figure 8.8: Confusion matrix of mDeBERTa-v3-base on IndicXNLI Telugu 62
Figure 8.9: Macro-F1 of the decision rule as the threshold changes 63
Figure 8.10: Start page 64
Figure 8.11: Report for an English rumour about 5G towers 65
Figure 8.12: Hovering a claim lights up its sentence in the article 66
Figure 8.13: Report for an article that mixes true and false claims 67
Figure 8.14: Report for a Telugu message 68
Figure 8.15: Image report for a generated image, with the Grad-CAM heatmap 69
Figure 8.16: Image report for a real photograph 69
Figure 8.17: Verify claims page 70
Figure 8.18: Batch check of three texts, with download 71
Figure 8.19: History of past checks 71
Figure 8.20: Report in the dark theme 72
Figure 8.21: Start page on a phone (375 px wide) 72
Figure 8.22: Interactive API documentation generated by FastAPI 73

LIST OF TABLES

Table 2.1: Summary of related work 9
Table 3.1: Functional requirements 14
Table 3.2: Non-functional requirements 15
Table 3.3: Hardware requirements 16
Table 3.4: Software requirements 16
Table 4.1: Structure of the checks table 27
Table 4.2: Backend API endpoints 28
Table 5.1: Datasets used in this project 31
Table 5.2: Dataset bias check with a TF-IDF and logistic regression model 33
Table 5.3: Preprocessing steps 34
Table 5.4: Training hyperparameters 39
Table 6.1: Technology stack 41
Table 7.1: Levels of testing 47
Table 7.2: Unit test cases (actual results from evaluation/run_unit_tests.py) 47
Table 7.3: Integration test cases (actual results from evaluation/run_test_cases.py) 49
Table 7.4: System test cases (actual results from evaluation/run_ui_tests.py) 51
Table 7.5: Measured response times on the development laptop (CPU only) 52
Table 7.6: Test summary 53
Table 8.1: Test results of the text and image classifiers 55
Table 8.2: Per-class results of the Telugu classifier 57
Table 8.3: Per-class results of the image detector 59
Table 8.4: Claim verification results (threshold $\tau = 0.5$) 60
Table 8.5: Three-class NLI on IndicXNLI Telugu (1,000 pairs) 62
Table 8.6: Two curated claims and their predictions 62
Table 8.7: Summary of results 74
Table 10.1: Launcher options 79
Table 10.2: Main configuration settings 80
Table 10.3: Common problems and solutions 82

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BERT	Bidirectional Encoder Representations from Transformers
BGE-M3	BAAI General Embedding, Multi-lingual, Multi-functionality, Multi-granularity
CLIP	Contrastive Language-Image Pre-training
CPU / GPU	Central / Graphics Processing Unit
CSV	Comma-Separated Values
DeBERTa	Decoding-enhanced BERT with disentangled Attention
DFD	Data Flow Diagram
FAISS	Facebook AI Similarity Search
FEVER	Fact Extraction and VERification (dataset)
Grad-CAM	Gradient-weighted Class Activation Mapping
LIME	Local Interpretable Model-agnostic Explanations
LR	Logistic Regression / Learning Rate
NLI	Natural Language Inference
NLLB	No Language Left Behind (translation model)
NLP	Natural Language Processing
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SPA	Single-Page Application
TF-IDF	Term Frequency – Inverse Document Frequency
UML	Unified Modelling Language
ViT	Vision Transformer
XAI	Explainable Artificial Intelligence

CHAPTER 1
1. INTRODUCTION

1.1 Overview

Soon after the 2000 rupee note was introduced in November 2016, messages spread across India claiming that it carried a GPS nano chip that let the government track black money. The note has no chip and the Reserve Bank of India never made such a claim, yet the story kept resurfacing in forwarded messages. Stories like this travel well because they are short, carry a confident tone and are passed on by people the reader trusts. A study of about 126,000 stories shared on Twitter between 2006 and 2017 found that false news reached 1,500 people roughly six times faster than true news [1].

Images now spread misinformation as easily as text. Diffusion models such as Stable Diffusion [20] produce photographs of events that never happened, and an image placed next to a false headline makes the headline more believable. A reader who wants to check a message today has to search for the claim, look for a fact-check article, and judge the picture by eye, usually in English, and usually one claim at a time.

TruthLens AI is a web application that does this checking in one place. A user pastes a news article in English or Telugu, or drops in an image, and the system returns:

- a verdict (fake or real for text, AI-generated or real for images) with the model's confidence,
- the words or image regions that led to the verdict,
- for text, each factual claim in the article marked Supported, Contradicted or No evidence found, together with the evidence and its source.

The last point is what separates this project from a plain classifier. A classifier that says "Fake, 91%" gives the reader nothing to verify. TruthLens also says which sentence is false and shows the passage that contradicts it, so the reader can judge the evidence for themselves.

1.2 Problem Statement

Existing fake news detectors have four gaps that make them hard to use for readers in Andhra Pradesh and Telangana:

1. Most are trained and tested only on English news, while a large share of local misinformation circulates in Telugu.
2. They check text or images, but rarely both in the same tool.
3. They give one label for a whole article and do not say which statement is false.
4. They do not explain their output, so the reader must either trust the model or ignore it.

The problem addressed in this project is to build a system that classifies English and Telugu news and detects AI-generated images, explains every verdict in a form a non-expert can read, and verifies the individual claims in an article against evidence, separating claims the evidence supports from claims it contradicts.

1.3 Motivation

Three observations shaped the design.

First, a verdict without a reason gives the reader nothing to check. If the system says only that a message is "fake", the reader has to decide whether to trust the model. If it shows the sentence that is false and the passage that contradicts it, the reader can judge the evidence directly and can see when the model is wrong. That is the reason the claim-level report is the centre of the user interface.

Second, transformer classifiers now reach very high accuracy on public fake news datasets, but much of that accuracy comes from style rather than truth. During this project a word-count model with no understanding of language scored 96.2% on the WELFake dataset after cleaning and 99.8% on a public Telugu dataset (Chapter 5). A system that only classifies style will fail on a carefully written false story. Checking claims against evidence does not depend on style.

Third, the tools needed to build such a system are now free. Every model in this project is open and downloaded from Hugging Face, all training data downloads without an account, and training runs on the free T4 GPUs in Google Colab and Kaggle. The full system runs on an ordinary laptop.

1.4 Objectives

The objectives of the project are:

1. To fine-tune DeBERTa-v3-large for English fake news classification and IndicBERT v2 for Telugu, and compare them with a BERT-base baseline and a TF-IDF model.
2. To build a Telugu training set whose two classes cannot be separated by writing style alone.
3. To detect AI-generated images with a linear classifier on CLIP ViT-L/14 features.
4. To explain text verdicts with LIME and image verdicts with Grad-CAM.
5. To design and evaluate a contradiction-aware, claim-level verification stage that labels each claim Support, Contradiction or Unknown and combines the labels into an overall credibility score.
6. To support batch checking of CSV files and image folders, with downloadable results.
7. To package the system so that it installs and starts on Windows, Linux or macOS with one command.

1.5 Scope of the Project

The system checks news text in English and Telugu. Other Indian languages are detected as English and are not supported by the classifiers, although the multilingual evidence retriever and NLI model would work for them with little change.

Claim verification can only confirm or contradict a claim when a passage about that claim exists in the evidence corpus or in the Google Fact Check Tools results. The project ships with a small hand-checked corpus of 40 passages on Indian current affairs, science and common health hoaxes, plus a script to add Wikipedia articles. Building a large, current evidence base is outside the scope of a final-year project and is listed as future work.

The image detector is trained on CIFAKE, which contains 32 by 32 pixel images from one generator. It shows that the approach works, but it is not a production detector for full-resolution images from current generators.

The system is a decision aid. It does not remove, block or report content, and its verdicts are meant to be read together with the evidence it shows.

1.6 Methodology in Brief

The project was carried out in five stages:

1. **Data preparation.** WELFake was cleaned of publisher names and its labels were corrected; a Telugu set was built by machine translation; CIFAKE was sampled and split.
2. **Model training.** All models were trained in one notebook on a free T4 GPU, with results written to disk for this report.
3. **Claim verification.** The claim extractor, evidence retriever, NLI verifier and aggregation rule were built from pre-trained models and evaluated on FEVER, Climate-FEVER, IndicXNLI and a curated claim set.
4. **Application.** A FastAPI backend and a React frontend were built around the models, with a launcher that sets everything up.
5. **Testing and evaluation.** Functional test cases were run against the live application, and every model was evaluated on held-out data.

1.7 Organisation of the Report

Chapter 2 reviews earlier work on fake news detection, Indian-language resources, AI-generated image detection, explainability and automated fact-checking. Chapter 3 analyses the existing systems, the proposed system and its requirements. Chapter 4 presents the system design with UML diagrams, data flow diagrams and the database design. Chapter 5 describes the datasets, models and algorithms in detail. Chapter 6 covers the implementation of the backend, frontend and training pipeline. Chapter 7 lists the test cases and their results. Chapter 8 reports and discusses the results of every model and of the claim verification stage. Chapter 9 concludes the report and lists future work, and Chapter 10 is a guide to installing and using the system.

CHAPTER 2
2. LITERATURE SURVEY

This chapter reviews the work that TruthLens builds on. It is organised by the five problems the system solves: classifying news text, handling Indian languages, detecting generated images, explaining model output, and verifying claims against evidence. The chapter ends with a comparison table and the gaps that this project addresses.

2.1 Fake News Detection from Text

2.1.1 Feature-based methods

Early detectors treated fake news as a text classification problem with hand-built features. Shu et al. [2] grouped these features into news content features (lexical, syntactic and visual) and social context features (who shared the story and how it spread), and argued that content alone is often not enough because fake stories are written to look like real ones. Rashkin et al. [42] compared the language of satire, hoaxes, propaganda and trusted news and found measurable differences in the use of first and second person pronouns, superlatives and subjective words. These findings explain why simple word-count models do well on many datasets, a point that returns in Chapter 5 of this report.

2.1.2 Benchmark datasets

Progress in the field has followed the release of public datasets. LIAR [3] contains about 12,800 short political statements from PolitiFact, each labelled on a six-point truthfulness scale. FakeNewsNet [44] collects full articles from PolitiFact and GossipCop together with social context such as user engagement. WELFake [4] merges four earlier corpora (Kaggle, McIntire, Reuters and BuzzFeed Political) into 72,134 labelled articles; its authors proposed combining word embeddings with linguistic features in a voting classifier. WELFake is large, balanced and free to download, which is why it is used for training in this project, but Chapter 5 shows that it has label and source shortcuts that must be removed before training.

2.1.3 Deep learning and transformers

Recurrent and convolutional networks replaced hand-built features around 2017, and transformers [34] replaced them in turn. BERT [6] is pre-trained by predicting masked words and is then fine-tuned on a labelled task. FakeBERT [5] combined BERT

embeddings with parallel one-dimensional convolutional layers and reported higher accuracy than earlier neural models on a Kaggle fake news set.

DeBERTa [7] changes BERT in two ways. Each word is represented by two vectors, one for content and one for position, and attention is computed from both separately, so the model can tell that "deep learning" and "learning deep" differ in meaning. DeBERTa-v3 [8] replaces masked-word prediction with ELECTRA-style replaced-token detection, where the model learns to spot words that a smaller generator has swapped in. DeBERTa-v3-large has about 435 million parameters and is one of the strongest openly available encoders for English classification, which is why the project abstract specifies it as the English model.

2.2 Indian-Language Resources

Labelled fake news data for Indian languages is scarce. BanFakeNews [9] provides about 50,000 Bangla news articles labelled authentic or fake, with baselines ranging from linguistic features to neural models. For Telugu, a dataset was published on Zenodo in 2024 [13]. Section 5.2 of this report examines it and shows that its two classes can be separated by writing style alone, which is why it was not used for training.

Multilingual encoders make Telugu classification possible without training a model from scratch. XLM-RoBERTa [37] was pre-trained on 100 languages. IndicBERT [10], from the AI4Bharat group, was pre-trained on Indian-language text and evaluated on the IndicGLUE benchmark. IndicBERT v2 [11] extends this to 23 Indian languages and English using the larger IndicCorp v2 corpus, and is the Telugu classifier in this project. For translation between English and Telugu, Meta's NLLB-200 [12] covers 200 languages with one model; its distilled 1.3 billion parameter version is used to build the Telugu training set.

2.3 AI-Generated Image Detection

Generative adversarial networks [35] were the first models to produce convincing fake faces, and diffusion models [36] such as latent diffusion [20] now produce photographs of almost any scene from a text prompt.

Wang et al. [16] trained a ResNet-50 on images from one GAN (ProGAN) with blur and JPEG augmentation and found that it detected images from several other GAN

architectures. Later work showed that this transfer breaks for diffusion models: Corvi et al. [18] found that detectors trained on GAN images performed poorly on diffusion images, and that diffusion models leave their own traces in the frequency spectrum.

Ojha et al. [17] took a different approach. Instead of training a network to find generator artefacts, they used the frozen image encoder of CLIP [14], a vision transformer [15] trained to match 400 million images with their captions, and trained only a nearest-neighbour or linear classifier on its features. Because CLIP's features were never tuned to one generator, this detector generalised to unseen generators, including diffusion models, better than fully trained networks. TruthLens follows this approach with CLIP ViT-L/14.

CIFAKE [19] pairs 60,000 real CIFAR-10 images with 60,000 images generated by Stable Diffusion v1.4 at the same 32 by 32 resolution. Its authors trained a convolutional network and used Grad-CAM to show that the network often relied on small details in the image background rather than the main object.

2.4 Explainable AI

LIME [21] explains a single prediction of any classifier. It creates many copies of the input with random parts removed, records how the model's output changes, and fits a small weighted linear model to those changes. The weights of the linear model are the explanation. For text, the parts are words, so LIME can say that removing "BREAKING" lowered the fake probability by a certain amount. LIME needs only the model's output, which makes it model-agnostic, but it needs hundreds of model runs per explanation.

Grad-CAM [22] explains a convolutional or transformer image model with a single backward pass. It takes the activations of a chosen layer, weights each channel by the gradient of the target class with respect to that channel, and sums them into a heatmap that shows which regions raised the class score. TruthLens uses LIME for text and Grad-CAM for images.

2.5 Automated Fact-Checking

Guo et al. [26] describe automated fact-checking as three steps: detecting claims worth checking, retrieving evidence, and predicting a verdict with a justification.

Claim detection. ClaimBuster [27] scores each sentence for check-worthiness, trained on sentences from US presidential debates labelled by human coders. Sentences with numbers, named entities and factual verbs score high and opinions score low. TruthLens uses a simpler rule-based score built on the same cues (Section 5.6).

Verdict prediction. FEVER [23] made verdict prediction a standard benchmark: about 185,000 claims written from Wikipedia sentences are labelled Supported, Refuted or NotEnoughInfo, with the evidence sentences that justify the label. Climate-FEVER [24] applies the same labels to 1,535 real claims about climate change collected from the web, which makes it harder and closer to real use. DeClarE [45] retrieved web articles for each claim and used attention over them to predict credibility.

Natural language inference. NLI asks whether a premise entails, contradicts or is neutral to a hypothesis. MNLI [28] and Adversarial NLI [29] are the main training sets. A model trained on NLI can verify a claim by treating the evidence as the premise and the claim as the hypothesis, without training on fact-checking data. Laurer et al. [30] showed that one NLI model can serve as a general classifier in this way, and released the DeBERTa-v3 NLI models used in TruthLens. For Telugu, IndicXNLI [25] provides NLI test pairs translated into Indian languages.

Explanations for fact-checks. Atanasova et al. [47] trained a model to produce a short written justification together with the verdict, and Kotonya and Toni [46] did the same for public health claims. TruthLens shows the evidence passage itself rather than a generated justification, so the reader sees exactly what the decision was based on.

Evidence retrieval. Dense retrieval embeds the claim and the passages in the same vector space and returns the nearest passages. BGE-M3 [31] produces such embeddings for more than 100 languages, including Telugu, and FAISS [32] searches millions of vectors quickly. Together they let TruthLens search the same evidence for English and Telugu claims.

2.6 Comparison of Existing Work

Table 2.1: Summary of related work

Work	Task	Method	Data	Limitation for this project
Shu et al. 2017 [2]	Fake news detection	Survey of content and social features	Various	Survey; no system

Work	Task	Method	Data	Limitation for this project
Wang 2017, LIAR [3]	Truthfulness rating	CNN with metadata	12.8K PolitiFact statements	Short statements, English only
Rashkin et al. 2017 [42]	Reliability of news	Linguistic analysis, LSTM	News and PolitiFact	Style cues, not facts
Verma et al. 2021, WELFake [4]	Fake news detection	Word embeddings + linguistic features	72,134 articles	Source shortcuts in data
Kaliyar et al. 2021, FakeBERT [5]	Fake news detection	BERT + 1D CNN	Kaggle fake news	No explanation, English only
Hossain et al. 2020, BanFakeNews [9]	Fake news detection	Traditional and neural models	About 50K Bangla articles	Bangla only
Doddapaneni et al. 2023, IndicBERT v2 [11]	Indian-language NLU	Multilingual pre-training	IndicCorp v2, 24 languages	General model, no fake news task
Wang et al. 2020 [16]	Generated image detection	ResNet-50 with augmentation	ProGAN and other GANs	Weak on diffusion images
Ojha et al. 2023 [17]	Generated image detection	Frozen CLIP + linear probe	GAN and diffusion images	Images only
Bird and Lotfi 2024, CIFAKE [19]	Generated image detection	CNN + Grad-CAM	120K images, 32 by 32	Low resolution, one generator
Ribeiro et al. 2016, LIME [21]	Explanation	Local linear surrogate	Model-agnostic	Slow (many model runs)
Selvaraju et al. 2017, Grad-CAM [22]	Explanation	Gradient-weighted activations	Image models	Images only
Hassan et al. 2017, ClaimBuster [27]	Claim detection	Supervised check-worthiness score	US debates	No verdict
Thorne et al. 2018, FEVER [23]	Fact verification	Retrieval + NLI baseline	185K Wikipedia claims	Written claims, English only

Work	Task	Method	Data	Limitation for this project
Popat et al. 2018, DeClarE [45]	Credibility	Attention over web evidence	Snopes, PolitiFact	No per-claim contradiction output
Laurer et al. 2024 [30]	Classification via NLI	DeBERTa-v3 NLI	MNLI, ANLI, FEVER-NLI, others	General method, not a system

2.7 Research Gaps

The review shows five gaps that TruthLens addresses:

1. **Language.** Strong fake news classifiers exist for English, but Telugu has no public training set whose classes differ in content rather than style.
2. **Modality.** Text and image detectors are built and evaluated separately, while the misinformation readers see often combines a headline with an image.
3. **Granularity.** Article-level classifiers do not say which statement is false. Fact verification systems do, but they are evaluated on benchmark claims and are rarely joined to a classifier in one tool.
4. **Contradiction.** Retrieval-based checkers that rank evidence by similarity cannot tell a passage that supports a claim from one that refutes it, because both are about the same subject. An explicit NLI step is needed.
5. **Explanation for readers.** Explanation methods are usually shown to researchers as heatmaps or weight lists. A reader needs the explanation inside the article: the influential words marked, the false sentence underlined, and the evidence printed next to it.

The next chapter analyses the existing approaches against these gaps and sets out the requirements of the proposed system.

CHAPTER 3
3. SYSTEM ANALYSIS

3.1 Existing System

Today a reader who wants to check a suspicious message has three options, and each one covers only part of the problem.

Manual fact-checking websites. Sites such as BOOM, Alt News, Factly and the PIB Fact Check unit publish articles that debunk specific viral claims. Their verdicts are reliable and come with sources, but each article covers one claim, takes hours or days to publish, and only exists for claims that became popular enough to be noticed. A reader must search for the claim and hope it has been checked.

Fake news classifiers. Research systems and some browser extensions classify an article as fake or real with a trained model. They answer instantly, but they give one label for the whole article, they are trained almost entirely on English, and they do not explain the label. As Chapter 5 shows, much of their accuracy comes from recognising the writing style of the sources in the training data.

Image checks. Reverse image search finds earlier copies of a photograph, which exposes old photos reused out of context, but it cannot say whether a new image was generated by a model. Separate AI-image detectors exist, but they are not connected to text checking.

3.1.1 Disadvantages of the existing system

- Verdicts are given for the whole article. The reader is not told which sentence is false.
- Classifiers do not explain their output, so a wrong verdict cannot be spotted.
- Retrieval tools that rank evidence by similarity cannot tell supporting evidence from contradicting evidence.
- Support for Telugu is limited to manual fact-check articles.
- Text and images are checked with different tools.
- Checking a list of messages means checking each one by hand.

3.2 Proposed System

TruthLens AI combines classification, explanation and claim-level evidence checking for English and Telugu text, and AI-generated image detection, in one web application.

For text, the article goes through two paths at once. The classifier path (DeBERTa-v3-large for English, IndicBERT v2 for Telugu) gives a fake probability, and LIME marks the words that pushed it. The verification path splits the article into claims, retrieves evidence for each one, and uses an NLI model to label each claim Support, Contradiction or Unknown. The two paths are fused into one fake risk, and the report shows the verdict, the highlighted article, the claims with their status, and the evidence behind each claim.

For images, a linear classifier on CLIP ViT-L/14 features says whether the image is a real photograph or AI-generated, and a Grad-CAM heatmap shows the regions that drove the decision.

Batch mode runs the same checks on a CSV file of texts or a set of images and returns a CSV of results. Every check is saved in a local history.

3.2.1 Advantages of the proposed system

- Each claim in an article gets its own status and evidence, so the reader knows what exactly is false.
- Contradicting evidence is detected explicitly through NLI, not inferred from similarity.
- Every verdict has an explanation: highlighted words for text, a heatmap for images.
- English and Telugu are handled in one interface, with automatic language detection.
- Text and images are checked in the same tool.
- Batch mode handles hundreds of items with one upload.
- All models are free and open. The system runs offline on a laptop once the models are downloaded.
- One command installs and starts the system on Windows, Linux or macOS.

3.3 Feasibility Study

3.3.1 Technical feasibility

Every component uses mature, open-source software. The models are available on Hugging Face under licences that allow academic use: MIT for DeBERTa-v3, IndicBERT v2, BGE-M3 and the NLI models, Apache 2.0 for BERT-base, OpenAI's MIT release for CLIP, and CC BY-NC 4.0 (non-commercial) for NLLB-200. PyTorch [38] and the Hugging Face Transformers library [33] load and fine-tune them with a few lines of code. Training DeBERTa-v3-large needs a GPU with about 16 GB of memory, which the free T4 in Google Colab and Kaggle provides; the notebook's full training run is planned to fit in one free session of 3.5 to 4.5 hours. Inference runs on a CPU, slowly for the largest model but within the limits of a demonstration. The project is technically feasible.

3.3.2 Economic feasibility

The project costs nothing beyond the students' existing laptops. All datasets download without payment or an account, all models are free, training uses free cloud GPUs, and the software stack (Python, FastAPI, React, SQLite) is open source. The optional Google Fact Check Tools API is free within its quota. The project is economically feasible.

3.3.3 Operational feasibility

The user interface needs no training: the Start page has one text box and one image drop area, and every report uses plain words such as "Likely fake" and "Contradicted". The launcher creates the Python environment, installs the packages, starts both servers and opens the browser, so the students and examiners do not need to know Python or Node.js to run it. The project is operationally feasible.

3.4 Requirement Specification

3.4.1 Functional requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The system shall accept news text of 10 to 20,000 characters in English or Telugu.
FR2	The system shall detect the language automatically and allow the user to override it.
FR3	The system shall classify text as Fake or Real with a confidence value.
FR4	The system shall highlight the words that most influenced the text verdict.

ID	Requirement
FR5	The system shall extract up to eight check-worthy claims from an article.
FR6	The system shall retrieve evidence for each claim and label it Support, Contradiction or Unknown.
FR7	The system shall show the evidence passage, its source and NLI scores for each claim.
FR8	The system shall combine claim labels into a credibility score and overall verdict.
FR9	The system shall fuse the classifier and claim results into one fake risk.
FR10	The system shall accept JPG, PNG and WEBP images up to 15 MB and classify them as Real or AI-generated.
FR11	The system shall produce a Grad-CAM heatmap for each image verdict.
FR12	The system shall check a CSV file of texts or a set or ZIP of images in one request.
FR13	The system shall let the user download batch results as a CSV file.
FR14	The system shall store every check in a history that can be viewed, filtered and deleted.
FR15	The system shall show whether each model is trained and loaded.
FR16	The system shall let the user save any report as a PDF.

3.4.2 Non-functional requirements

Table 3.2: Non-functional requirements

Category	Requirement
Usability	Plain-language verdicts; one input box on the Start page; works with keyboard only; supports screen readers.
Performance	Verdict and claim results are shown before the slower LIME explanation, which is loaded separately.
Portability	Runs on Windows 10/11, Ubuntu and macOS with Python 3.10 to 3.12 and Node.js 18 or later.
Reliability	Missing or untrained models return a clear message instead of an error; finished models are never retrained by mistake.
Privacy	All processing is local. No text or image leaves the machine unless the optional Fact Check API key is set.
Maintainability	Models, thresholds and evidence folder are set in one configuration file; evidence is added by dropping JSONL files in a folder.
Accessibility	Colour is never the only signal: verdicts are also written as words and claim status also uses underline style.
Responsiveness	The interface works from 360 px phone screens to desktop monitors, with light and dark themes.

3.4.3 Hardware requirements

Table 3.3: Hardware requirements

Component	Minimum (running the app)	Recommended / training
Processor	4-core 64-bit CPU	8 or more cores
Memory	8 GB RAM (text or image models, not all at once)	16 GB RAM
Storage	15 GB free (models and environment)	25 GB free
GPU	Not required	NVIDIA GPU with 16 GB (free T4 on Colab or Kaggle for training)
Network	Needed once to download models	Needed for training notebook

3.4.4 Software requirements

Table 3.4: Software requirements

Layer	Software
Operating system	Windows 10/11, Ubuntu 22.04 or later, macOS 13 or later
Languages	Python 3.10 to 3.12, JavaScript (ES2022)
Backend	FastAPI, Uvicorn, SQLAlchemy, Pydantic Settings
Machine learning	PyTorch 2, Transformers 4.4x, Sentence-Transformers, FAISS, LIME, pytorch-grad-cam, spaCy, scikit-learn
Frontend	React 18, React Router 6, Vite 6, Tailwind CSS 4, Axios
Database	SQLite
Training	Google Colab or Kaggle notebook (T4 GPU), Hugging Face Hub
Tools	VS Code, Git, a modern browser (Chrome, Edge, Firefox)

3.5 Software Development Model

The project followed an iterative model. A first version with all three pipelines and a basic interface was built and tested end to end with tiny placeholder models. The training notebook was then developed and run in a quick-test mode that trains every model on a few hundred examples in about 25 minutes, which caught problems such as the WELFake label inversion before the full run. The interface was redesigned twice after review. Each iteration ended with the application running and the test cases in Chapter 7 passing.

CHAPTER 4
4. SYSTEM DESIGN

This chapter describes how TruthLens AI is put together: its architecture, its modules, the UML diagrams that show how the parts interact, the data flow, the database and the programming interface between the frontend and the backend.

4.1 System Architecture

TruthLens follows a three-tier architecture. The presentation tier is a React single-page application served by Vite on port 5173. The application tier is a FastAPI service on port 8000 that holds all the machine learning models. The data tier is a SQLite database for the check history and a folder of JSON-lines files with a FAISS index for the evidence corpus. The browser never talks to the backend directly: Vite forwards every request under `/api` to port 8000, so the frontend and backend share one origin and no cross-origin configuration is needed on the user's machine.

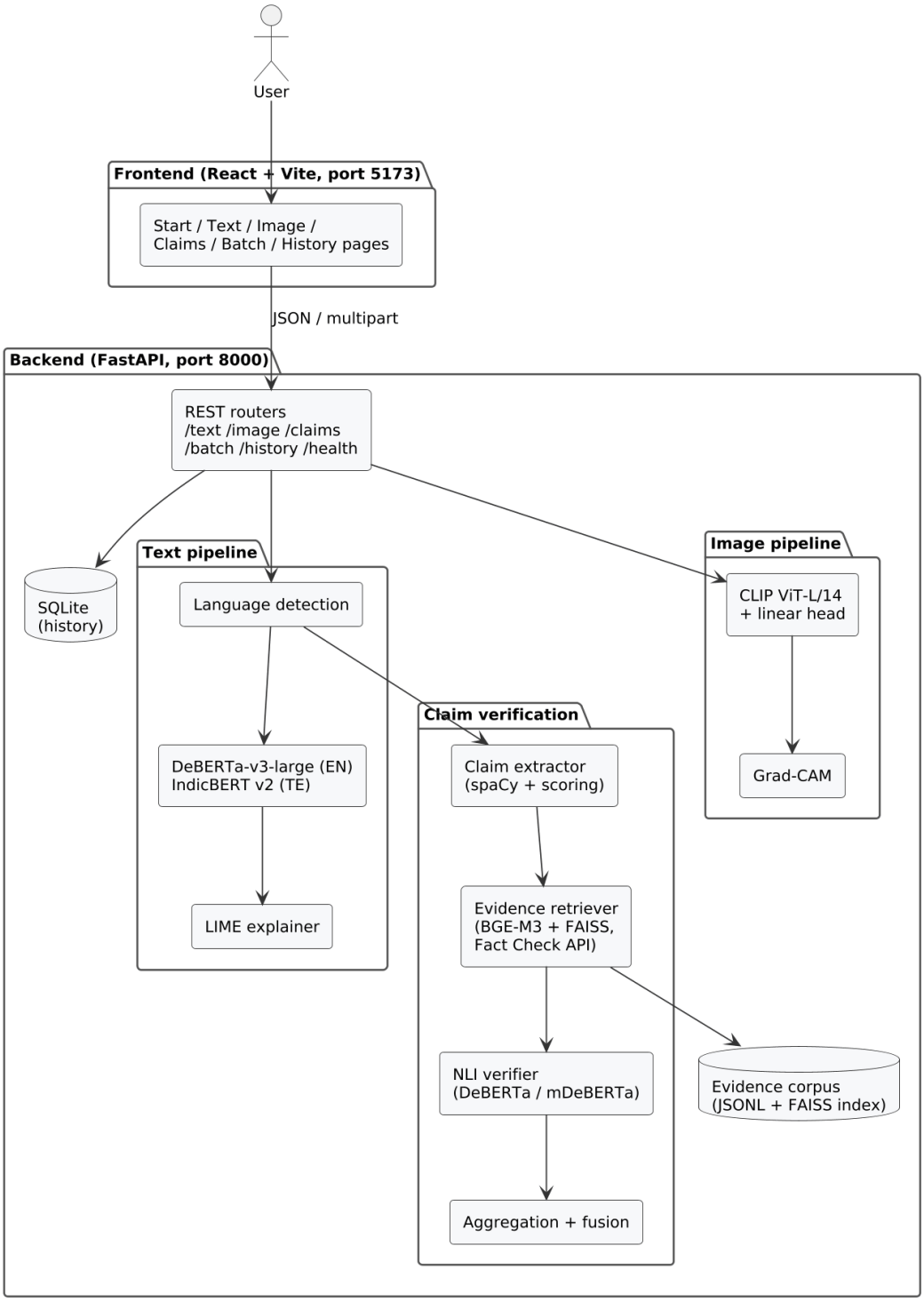


Figure 4.1: Overall architecture of TruthLens AI

Inside the backend, the three pipelines are independent Python modules. Each model is wrapped in a `Lazy` object that loads it on the first request and keeps it in memory afterwards, guarded by a lock so that two simultaneous requests do not load the same model twice. This keeps start-up fast and means a user who only checks images never loads the 1.7 GB English text model.

4.2 Module Description

4.2.1 User interface module

The interface has six pages, all reachable from a tool rail on the left (a top bar on phones):

- **Start.** One text box and one image drop area. Text is sent to the text check, an image to the image check.
- **Check news text.** The input form on the left and the report on the right: verdict and fake-risk scale, the article with highlighted words and underlined claims, and the claim list with evidence.
- **Check an image.** Image preview, verdict scale, and a slider that wipes between the original image and its Grad-CAM heatmap.
- **Verify claims.** Claim verification on its own, without the classifier.
- **Check many at once.** Upload a CSV of texts or a set of images and download the results.
- **History.** Every past check with its result, filterable by type.

Results are kept in memory when the user moves between pages, so a report is not lost by switching tools.

4.2.2 Language detection module

Telugu letters occupy the Unicode block U+0C00 to U+0C7F. The module counts the letters in the text and treats it as Telugu when more than 30% fall in that block. The threshold allows English words, numbers and names inside a Telugu article.

4.2.3 Text classification module

This module loads the fine-tuned English or Telugu model, tokenises the text to at most 256 tokens and returns the probability of each class. It reads the label names from the model's own configuration file, so a model trained with a different label order still reports Fake and Real correctly.

4.2.4 Explanation module

The text explainer runs LIME with 200 perturbed copies of the article and returns a weight for every word. The image explainer runs Grad-CAM on the last transformer block of CLIP and returns the heatmap blended with the original image as a PNG.

4.2.5 Claim verification module

This module has four parts: the claim extractor, the evidence retriever, the NLI verifier and the aggregator. They are described as an algorithm in Section 5.6 and shown as an activity diagram in Figure 4.6.

4.2.6 Image detection module

The image detector resizes and centre-crops the image to 224 by 224 pixels, passes it through the frozen CLIP ViT-L/14 encoder, and applies the trained linear head to the 768-dimensional embedding.

4.2.7 Batch module

The batch module reads a CSV file with a `text` column (and an optional `language` column), or a list of images, or a ZIP archive of images. It runs the same models row by row, writes a result CSV under a random job identifier, and returns a summary with counts per label.

4.2.8 History module

Every check is saved in the `checks` table with its type, a preview of the input, the verdict and the confidence. The history page lists, filters and deletes these records.

4.3 UML Diagrams

4.3.1 Use case diagram

The use case diagram shows the two kinds of actor. A user checks text, images and claims, runs batch checks and manages the history. An administrator, in practice a student maintaining the project, trains the models and adds evidence documents. Highlighting words extends the text check because it can be switched off; claim verification is included in it by default.

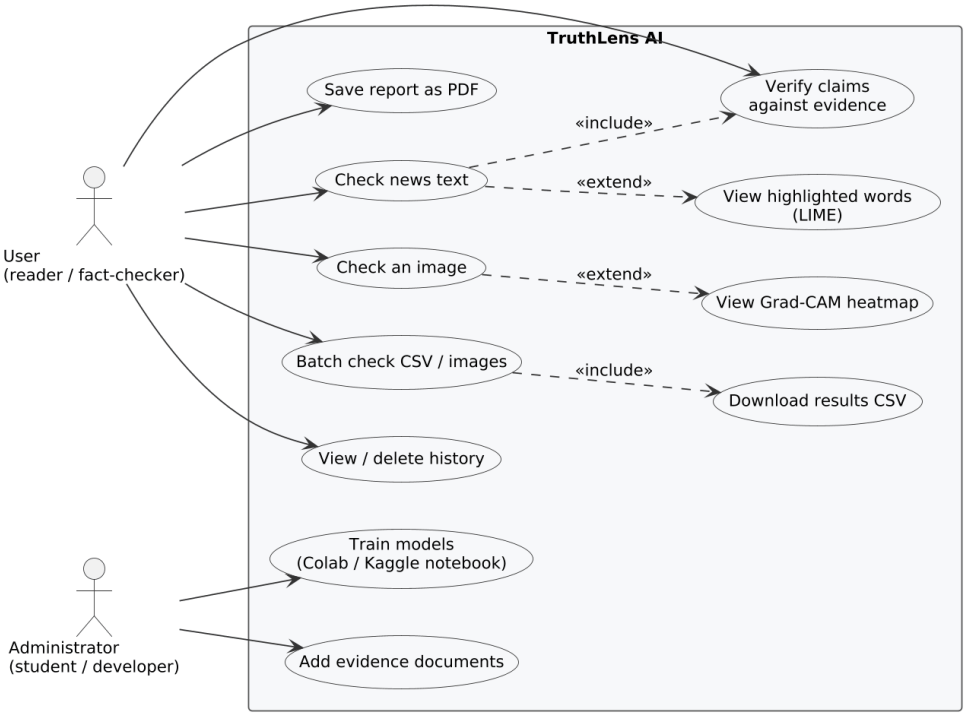


Figure 4.2: Use case diagram

4.3.2 Class diagram

The class diagram shows the main backend classes. `Settings` holds every configurable value, read from the `.env` file. `Lazy` wraps each model so it loads once. `TextClassifier`, `ImageDetector`, `NLIModel` and `EvidenceRetriever` each wrap one model, and the claim verifier module uses the last two. `CheckRecord` is the database table.

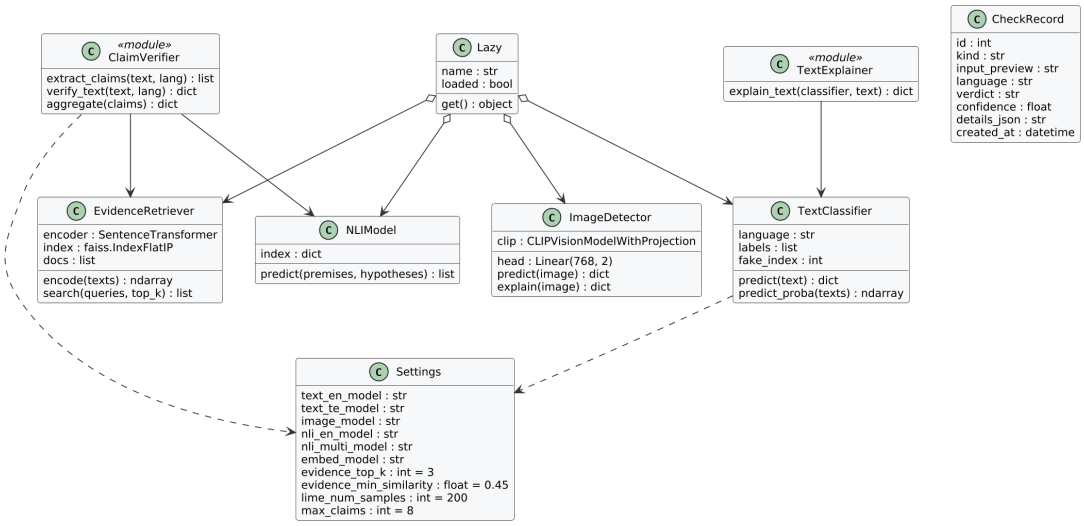


Figure 4.3: Class diagram of the backend

4.3.3 Sequence diagram: checking news text

The sequence diagram for a text check shows the two-step response. The first request returns the verdict and the claim evidence. A second request computes the LIME explanation and the highlights are added to the report already on screen, so the user is not kept waiting for the slowest step.

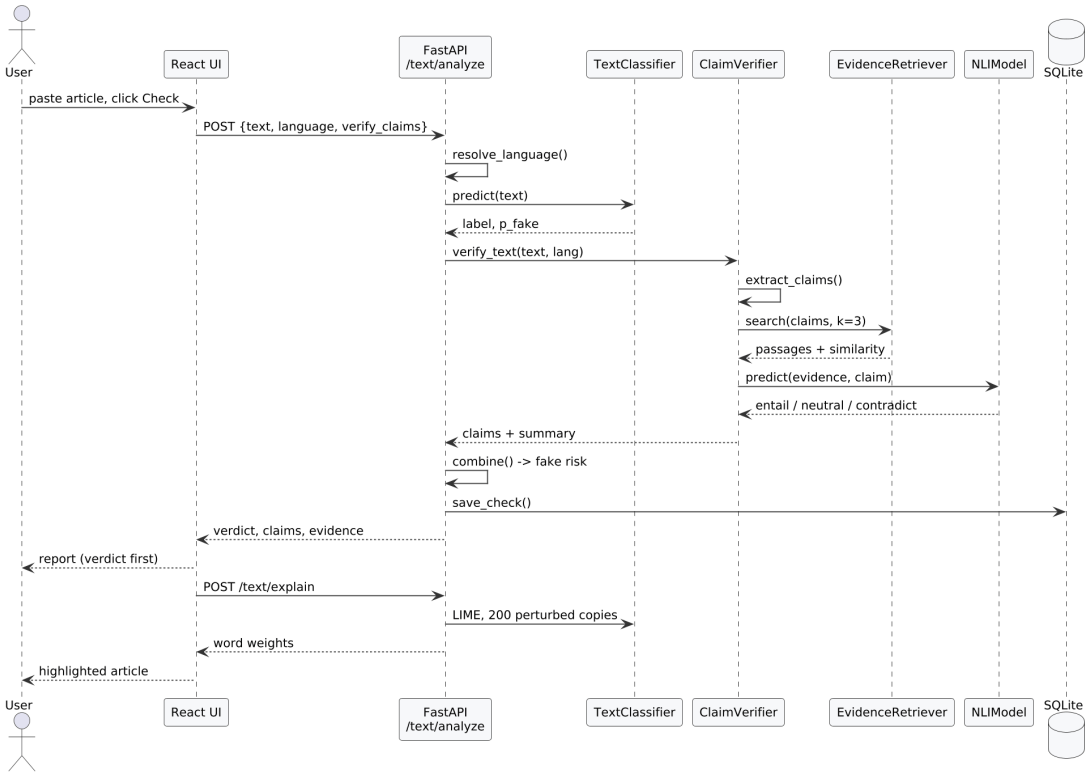


Figure 4.4: Sequence diagram for checking news text

4.3.4 Sequence diagram: checking an image

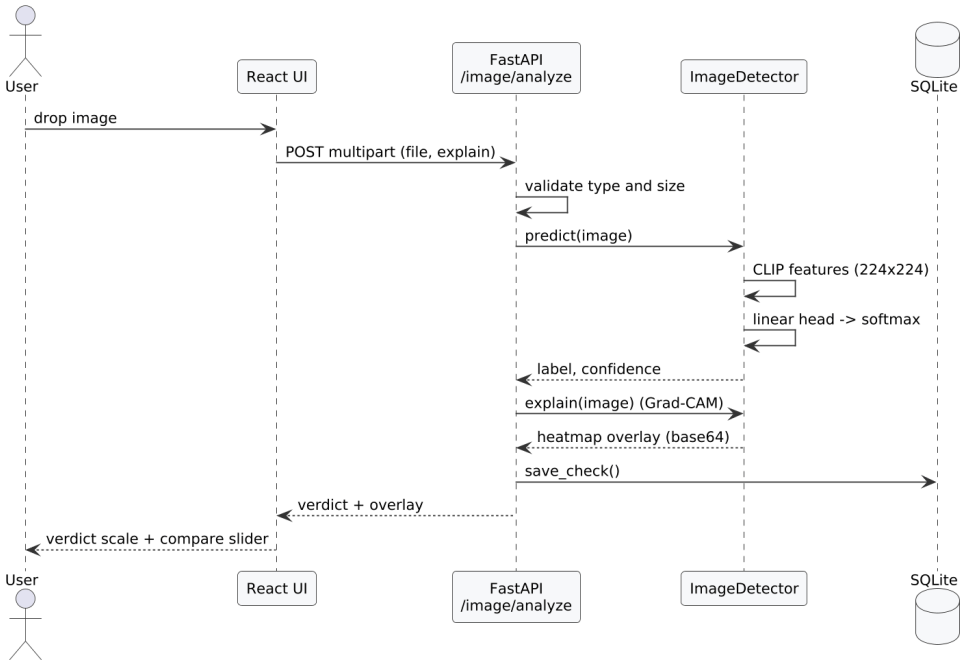


Figure 4.5: Sequence diagram for checking an image

4.3.5 Activity diagram: claim verification

The activity diagram follows one article through claim verification. The decision on contradiction comes before the decision on support, so a passage that clearly refutes a claim is not outweighed by passages that only partly agree with it.

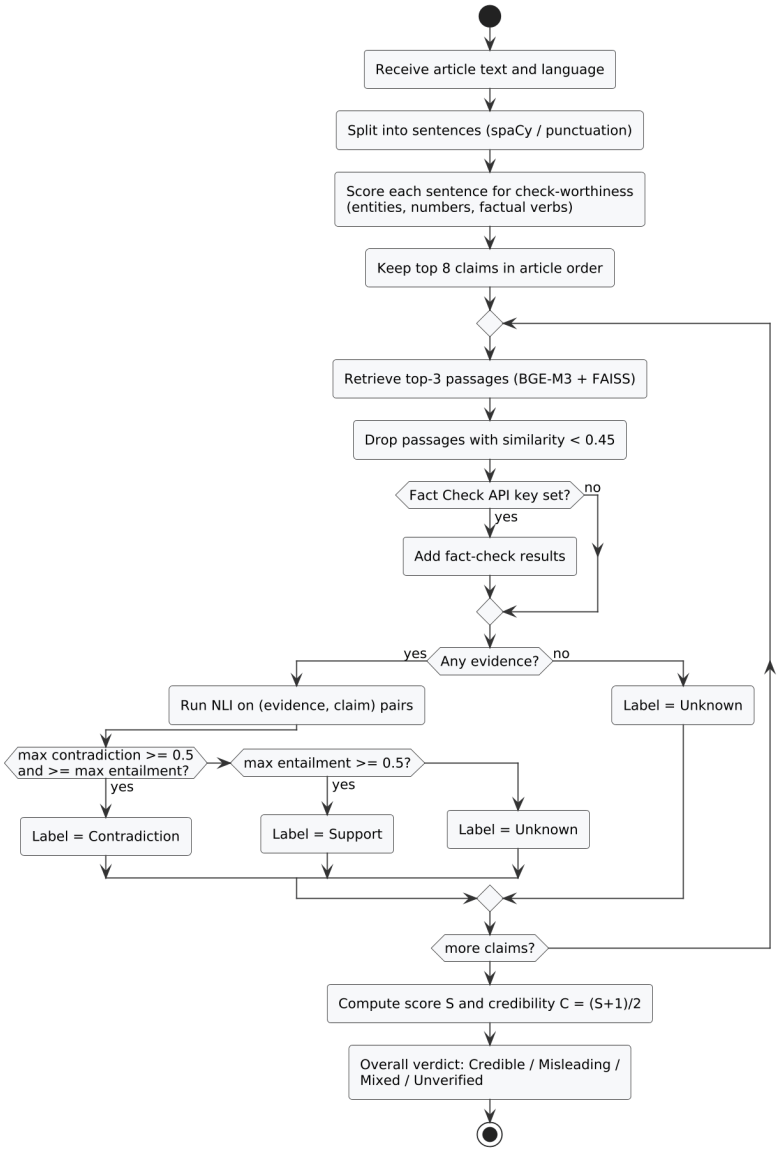


Figure 4.6: Activity diagram of claim verification

4.3.6 Activity diagram: overall flow

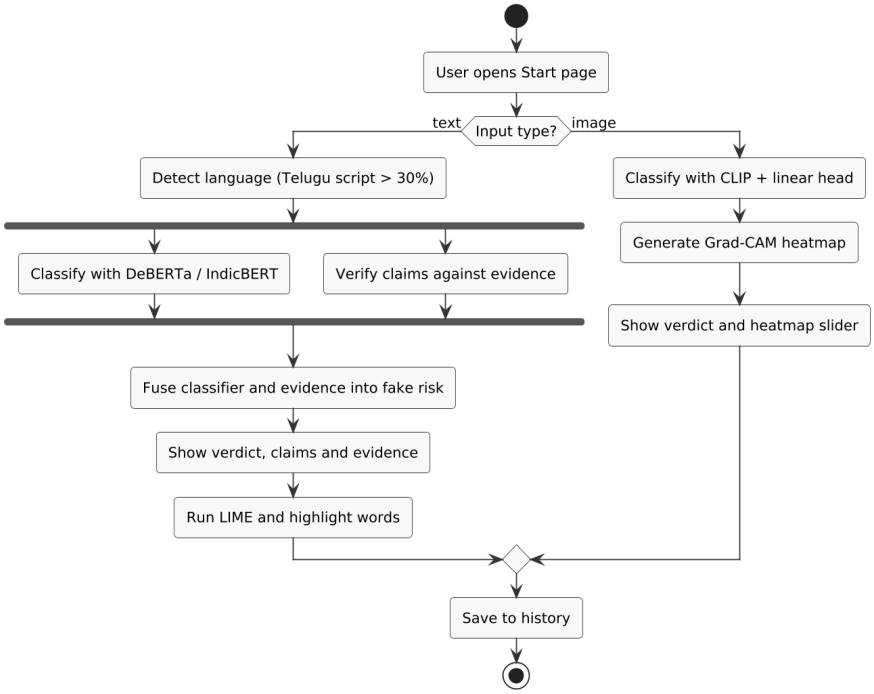


Figure 4.7: Activity diagram of the overall check

4.3.7 Deployment diagram

The deployment diagram shows where each part runs. Training runs once in a cloud notebook and produces a ZIP of model files. Everything else runs on the student's laptop: the browser, the Vite server and the Python environment with the models and the database.

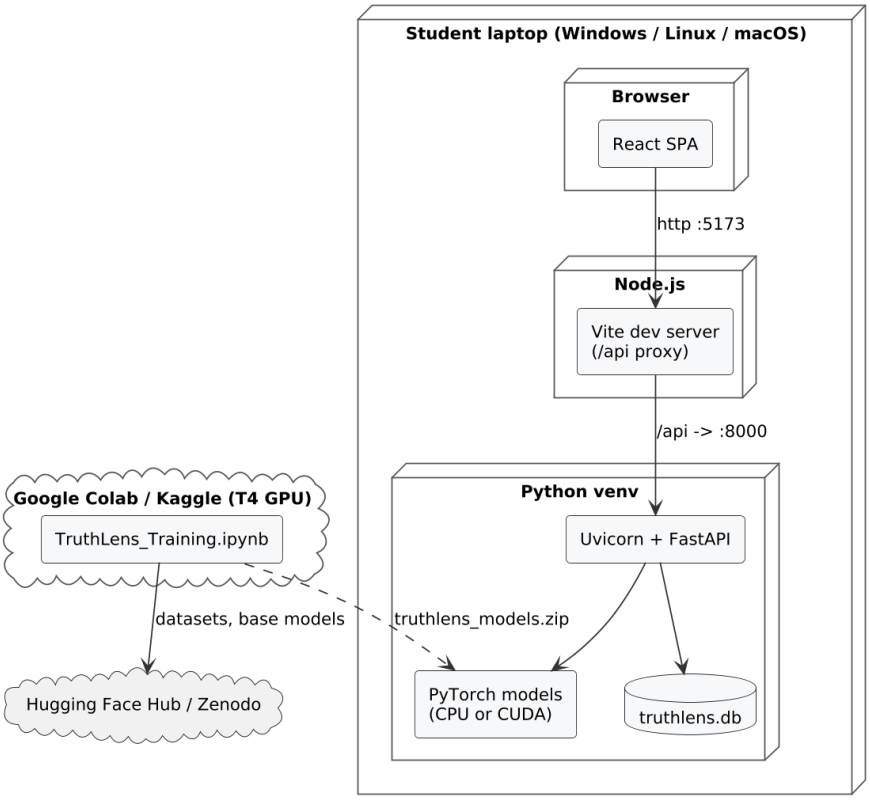


Figure 4.8: Deployment diagram

4.4 Data Flow Diagrams

The level 0 data flow diagram treats TruthLens as a single process. The user sends text, images or a CSV file and receives verdicts, explanations and evidence. Evidence sources and trained models are external inputs.

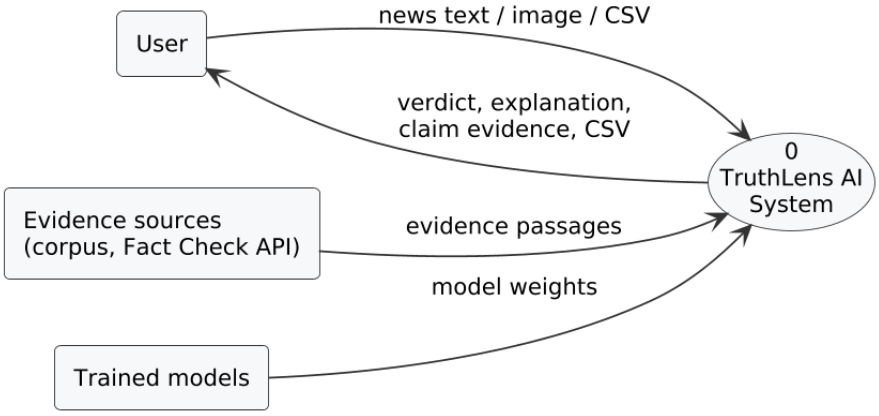


Figure 4.9: Level 0 data flow diagram

The level 1 diagram splits the system into its six processes. Language detection feeds both the classifier and the claim verifier. The fusion process combines their outputs and writes the result to the history store. Image detection runs on its own path.

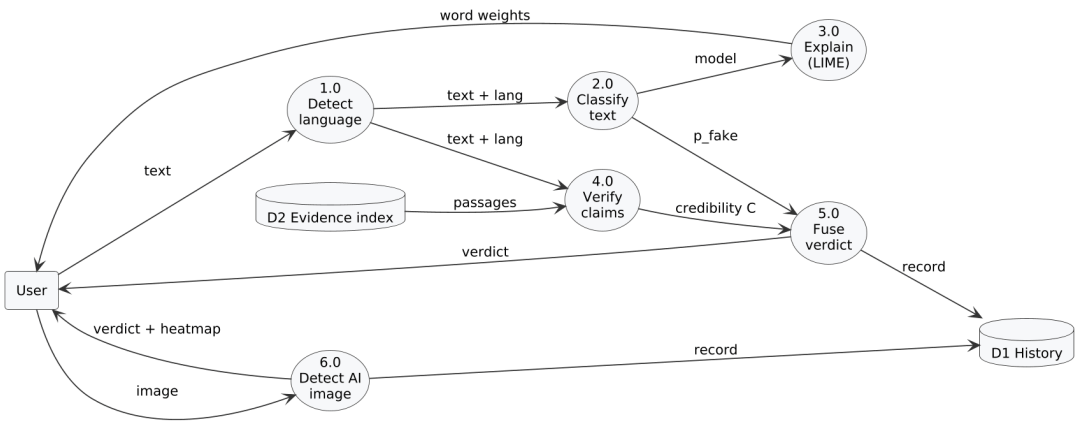


Figure 4.10: Level 1 data flow diagram

4.5 Database Design

The system has one relational table. Evidence and batch results are stored as files rather than tables, because evidence is maintained by hand as text files and batch results are meant to be downloaded.

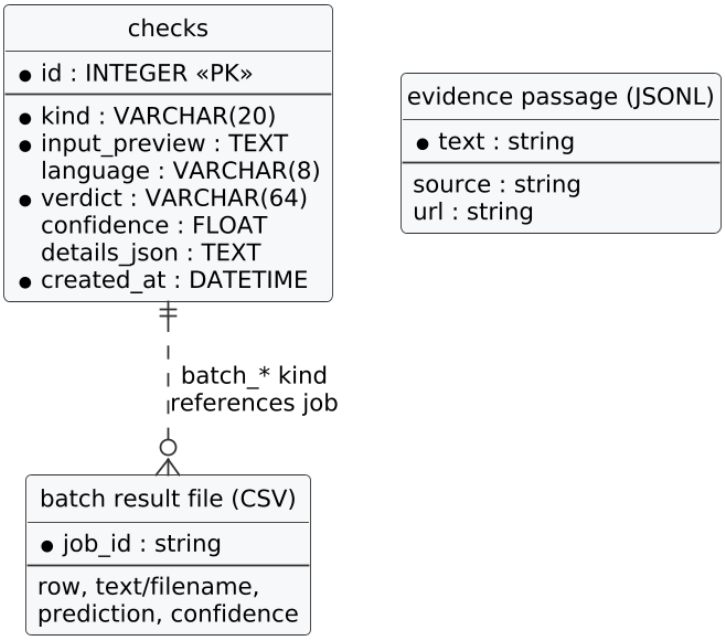


Figure 4.11: Entity relationship diagram

Table 4.1: Structure of the checks table

Column	Type	Constraint	Description
id	INTEGER	Primary key, auto-increment	Record identifier
kind	VARCHAR(20)	Not null, indexed	text, image, claims, batch_text or batch_image

Column	Type	Constraint	Description
input_preview	TEXT	Not null	First part of the text, or the image file name
language	VARCHAR(8)	Nullable	en or te for text checks
verdict	VARCHAR(64)	Not null	Final label shown to the user
confidence	FLOAT	Nullable	Confidence of the verdict, 0 to 1
details_json	TEXT	Default "{}"	Extra details such as probabilities and claim counts
created_at	DATETIME	Not null	Time of the check (UTC)

Each evidence passage is one line of JSON with three fields, so students can add evidence with any text editor. The example below is wrapped to fit the page; in the file it is a single line:

```
{"text": "Indian 2000 rupee banknotes do not contain any GPS or nano chip; viral claims
          about a tracking chip were false.",
 "source": "Wikipedia",
 "url": "https://en.wikipedia.org/wiki/Indian_2000-rupee_note"}
```

4.6 Interface Design

4.6.1 REST API

Table 4.2: Backend API endpoints

Method	Endpoint	Input	Output
POST	/api/text/analyze	text, language, explain, verify_claims	classification, verification, overall verdict
POST	/api/text/explain	text, language	LIME tokens and top words
POST	/api/claims/verify	text, language	claims with labels and evidence, summary
POST	/api/image/analyze	image file, explain	label, confidence, Grad-CAM overlay
POST	/api/batch/text	CSV file, language, verify_claims	job id, counts, results
POST	/api/batch/images	image files or ZIP	job id, counts, results
GET	/api/batch/{job_id}/download	job id	result CSV file
GET	/api/health	none	model status and metrics
GET	/api/history	limit, kind	list of past checks

Method	Endpoint	Input	Output
DELETE	/api/history/{id}	record id	confirmation
DELETE	/api/history	none	confirmation

4.6.2 Response format

Every text check returns the same JSON structure, which the frontend uses to draw the report. The example below is a real response, shortened, for the input "The new 2000 rupee notes contain a GPS nano chip that lets the government track them, officials confirmed on Monday." with highlighting switched off:


```
{
  "language": "en",
  "classification": {"label": "Real", "confidence": 0.9998,
                     "fake_probability": 0.0002},
  "verification": {
    "claims": [
      {"claim": "The new 2000 rupee notes contain a GPS nano chip that ...",
       "label": "Contradiction", "confidence": 1.0,
       "evidence": [{"text": "Indian 2000 rupee banknotes do not contain any GPS ...",
                     "source": "Wikipedia", "similarity": 0.712,
                     "nli": {"entailment": 0.0, "neutral": 0.0,
                             "contradiction": 1.0}}]}
    ],
    "summary": {"verdict": "Likely Misleading", "credibility_score": 0.0001,
                "counts": {"Support": 0, "Contradiction": 1, "Unknown": 0}}
  },
  "overall": {"verdict": "Fake", "fake_risk": 0.5001,
              "based_on": ["classifier", "claim_evidence"]}
}
```


The classifier and the evidence disagree completely here, and Section 8.6 discusses what this example shows.

4.6.3 User interface design

The interface was designed around the idea of a fact-checker's marked-up proof sheet. The article in a report is set in a serif typeface, like a newspaper. Influential words get highlighter-pen marks, red for words that push toward fake and green for words that push toward real, with stronger colour for more influence. Claims are underlined: a solid line for supported, a wavy line for contradicted and a dotted line for no evidence. Hovering a claim in the evidence list lights up its sentence in the article. Colour is never the only signal, because every verdict is also written in words.

CHAPTER 5
5. METHODOLOGY

This chapter explains how each part of TruthLens works: the data the models learn from, how the data was cleaned, the models and their training, the explanation methods, the claim verification algorithm, and the measures used to evaluate everything.

5.1 Overview

The methodology has two halves. The trained half covers three classifiers (English text, Telugu text and images) that need labelled data and a GPU to train. The pre-trained half covers claim verification, which uses an embedding model and NLI models exactly as they were released, with no training in this project. The two halves meet in the fusion step, which combines a classifier's fake probability with the credibility of the article's claims.

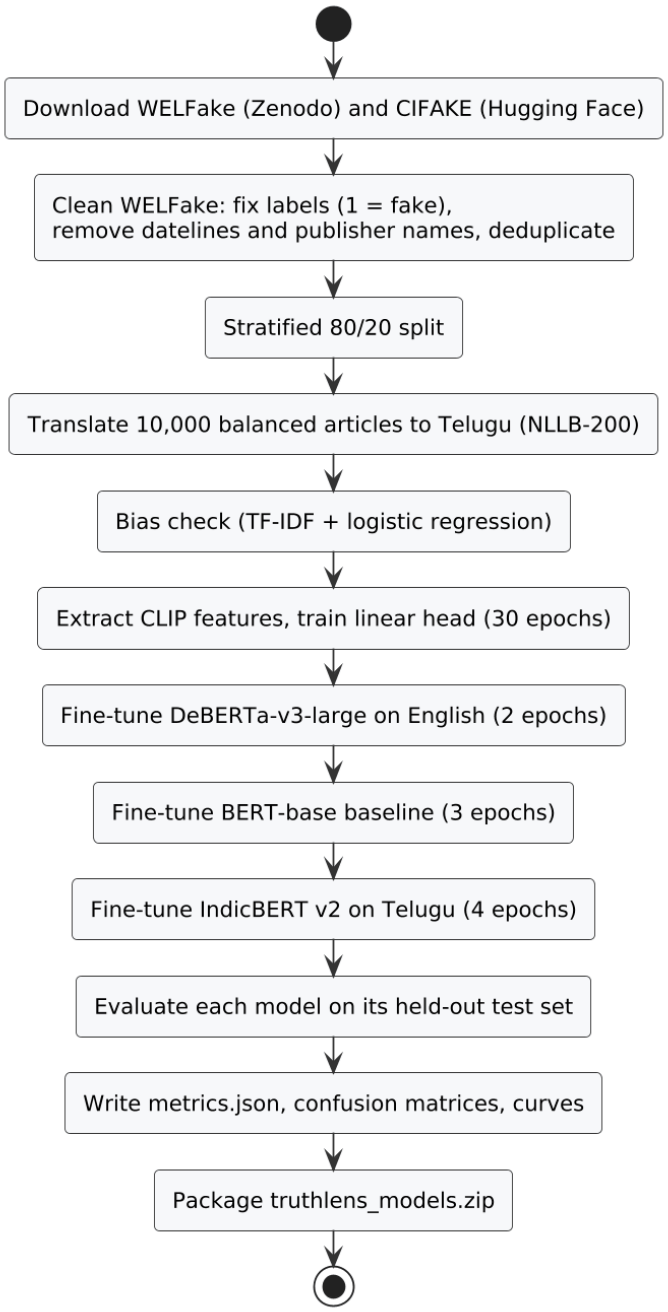


Figure 5.1: Training pipeline run in the notebook

5.2 Datasets

Table 5.1: Datasets used in this project

Dataset	Purpose	Source and licence	Size used
WELFake [4]	English classifier	Zenodo 4561253, CC BY 4.0	20,000 train, 12,727 test articles
WELFake-Telugu (built here)	Telugu classifier	Translated from WELFake with NLLB-200	8,000 train, 2,000 test articles

Dataset	Purpose	Source and licence	Size used
CIFAKE [19]	Image detector	Hugging Face, MIT	40,000 train, 10,000 test images
FEVER [23] dev	Claim verification test	Hugging Face (gold evidence)	900 claims
Climate-FEVER [24]	Claim verification test	Hugging Face	450 claims
IndicXNLI [25] Telugu	Telugu NLI test	Hugging Face	1,000 pairs
Curated claims (built here)	End-to-end test	Written for this project	40 claims

5.2.1 WELFake

WELFake has 72,134 news articles with a title, a body and a binary label. It was chosen because it is large, roughly balanced and downloads without an account. Two problems had to be fixed before it could be used.

The labels are the reverse of the documentation. The dataset description says label 0 is fake. Reading the rows shows the opposite: articles from Reuters and The New York Times carry label 0, and articles beginning with "BREAKING" or ending with "Featured image via" carry label 1. A first quick training run that followed the documentation produced a model that called Reuters reports fake. All later runs use 1 for fake.

Publisher names give the label away. About 60% of the real articles begin with a dateline such as "WASHINGTON (Reuters) -". Fake articles often end with credits such as "Featured image via Getty Images" or "Follow 21WIRE on Twitter". A model can reach high accuracy by learning these strings alone and would then fail on any article without them. The cleaning step removes datelines, the words "Reuters" and "The New York Times", photo credits, "21WIRE" and "Breitbart" tags, and "Follow ... on Twitter" lines, then drops duplicate and empty rows. About 63,600 articles remain.

The cleaned data is split 80/20 with the class balance kept the same in both parts (stratified). For training, 20,000 articles are drawn from the 80% part, and 10% of those (2,000) are held back for validation. The test set is the full 20% part, 12,727 articles, which the model never sees during training.

5.2.2 Telugu: why the public dataset was rejected

The obvious choice for Telugu was the dataset published on Zenodo in 2024 [13]. Before training on it, a simple test was run: a TF-IDF word-count model with logistic regression, which understands nothing about meaning, was trained to separate its two classes. It scored 99.8%.

The strongest features explained why. The fake class consists of fact-check reports machine-translated into Telugu, and the real class of native Telugu health and lifestyle articles. The words that best predicted "fake" were translation-style function words such as మరియు ("and") and యొక్క ("of"), and the name of the fact-checking site బూమ్ (BOOM). The words that best predicted "real" were health terms such as విటమిన్ (vitamin) and గుండె (heart). A model trained on this data would learn to recognise translated text and health articles, not false ones.

5.2.3 Telugu: building a translated set

To give both classes the same writing style, a balanced sample of 10,000 articles (5,000 fake and 5,000 real) was taken from the WELFake training part and translated with NLLB-200 [12] (distilled, 1.3 billion parameters). Each article was cut to its headline and leading sentences, at most 120 words, and translated one sentence at a time, which NLLB handles more reliably than long passages. The script saves progress every few batches, so a disconnected session resumes where it stopped.

The same TF-IDF test on this set scores 87.1%, down from 99.8%. It does not fall to 50% because the topics of WELFake carry over: the strongest fake-class features are ట్రంప్ (Trump), హిల్లరీ (Hillary) and "(వీడియో)" ("(video)"), and the strongest real-class features are reporting verbs such as చెప్పారు ("said") and weekday names. The Telugu model therefore has to learn more than vocabulary to beat 87.1%. The set is translated, not native Telugu, and its articles are about United States politics, which is a limitation discussed in Chapter 8.

Table 5.2: Dataset bias check with a TF-IDF and logistic regression model

Dataset	Rows used	Accuracy	F1	Strongest fake-class words	Strongest real-class words
Zenodo Telugu [13]	Full set	99.8%	n/a	మరియు, యొక్క, బూమ్	విటమిన్, గుండె
WELFake (cleaned)	30,000	96.2%	0.957	[video], hillary, october, 2016	said, u.s., president, minister

Dataset	Rows used	Accuracy	F1	Strongest fake-class words	Strongest real-class words
WELFake-Telugu (ours)	10,000	87.1%	0.875	ట్రంప్, హిల్లరీ, (వీడియో)	చెప్పారు, మంగళవారం, అధ్యక్షుడు

5.2.4 CIFAKE

CIFAKE contains 32 by 32 pixel images: real photographs from the CIFAR-10 dataset and images generated by Stable Diffusion v1.4 [20] to look like the same ten classes. Its training split has 50,000 images of each kind. This project samples 25,000 of each and holds out 20%, giving 40,000 training and 10,000 test images. Using half the training split keeps feature extraction inside one free GPU session.

5.3 Preprocessing

Table 5.3: Preprocessing steps

Data	Step	Reason
WELFake	Join title and body	Headlines carry much of the signal
WELFake	Set label 1 = fake	Labels in the file are reversed from the documentation
WELFake	Remove datelines, publisher names and credits	Prevents learning the source instead of the content
WELFake	Drop empty and duplicate texts	Avoids the same article in train and test
WELFake	Stratified 80/20 split, seed 42	Same class balance in train and test; repeatable
Telugu	Cut to headline + 120 words, translate by sentence	NLLB is more accurate on short inputs
All text	Tokenise to at most 256 tokens	Fits a T4 GPU with batch size 8 for the large model
Images	Resize, centre-crop to 224 by 224, CLIP normalisation	Input format CLIP was trained on

5.4 Text Classification Models

5.4.1 DeBERTa-v3-large for English

DeBERTa-v3-large [8] has 24 transformer layers, a hidden size of 1,024 and about 435 million parameters, of which 131 million are in its 128,000-word vocabulary

embedding. A classification head (one linear layer over the first token's representation) is added and the whole network is fine-tuned.

In standard self-attention every token is a single vector that mixes what the word is and where it is. DeBERTa keeps two vectors per token, content and relative position, and computes attention from three terms: content to content, content to position and position to content. For a query token i and key token j with content vectors H and relative position embeddings P:

$$A(i, j) = H_i H_j^T + H_i P_{ij}^T + P_{ji} H_j^T \quad (5.1)$$

The classifier turns the head's output z into class probabilities with the softmax function, and training minimises cross-entropy over the training set:

$$p(c \mid x) = \exp(z_c) / \Sigma_k \exp(z_k), \quad L = -(1/N) \Sigma_i \log p(y_i \mid x_i) \quad (5.2)$$

5.4.2 BERT-base baseline

BERT-base-uncased [6] (12 layers, 110 million parameters) is fine-tuned on the same data with the same pipeline. It is not used by the application; it shows how much the larger, newer model adds.

5.4.3 TF-IDF baseline

The TF-IDF model treats each article as a bag of words. Each word w in article d gets a weight

$$tfidf(w, d) = tf(w, d) \cdot \log(N / df(w)) \quad (5.3)$$

where tf is the word's count in the article, N the number of articles and df the number of articles containing the word. A logistic regression model is trained on these weights. It is used only to measure how much of each dataset can be solved without understanding language.

5.4.4 IndicBERT v2 for Telugu

IndicBERT v2 [11] (the MLM-only checkpoint) is a BERT-style encoder pre-trained on IndicCorp v2, a large corpus covering 23 Indian languages and English. It is fine-tuned on the translated Telugu set in the same way as the English model.

5.5 Image Detection Model

Following Ojha et al. [17], the CLIP ViT-L/14 image encoder [14] is frozen and only a linear classifier is trained on its output. The encoder cuts a 224 by 224 image into a 16

by 16 grid of 14-pixel patches, runs 24 transformer layers over the 256 patch tokens and a class token, and projects the class token to a 768-dimensional embedding.

For an image with embedding e, the detector computes

$$p = softmax(W e + b), \quad W \in \mathbb{R}^{2x768} \tag{5.4}$$

so only 1,538 numbers are learned. The embeddings of all 50,000 images are computed once and cached, and the head is then trained for 30 epochs with AdamW in a few minutes. Because CLIP's features were learned from 400 million image and caption pairs and were never tuned to one generator, the approach is less likely to rely on artefacts specific to the training generator than a fully trained network.

5.6 Claim Verification Algorithm

Claim verification uses no trained component from this project. It is built from an embedding model, an index and two NLI models, joined by an explicit decision rule.

5.6.1 Step 1: claim extraction

The article is split into sentences, with spaCy for English and punctuation for Telugu (the full stop and the Devanagari danda, which Telugu text sometimes uses). A sentence is dropped if it is a question, starts with an opinion or call to action ("I think", "share this", "click"), has fewer than five words (three for Telugu) or more than 80. Each remaining sentence s gets a check-worthiness score from the cues ClaimBuster [27] found useful:

$$w(s) = 1 + 0.8 \cdot \min(e, 3) + 1[\text{number}] + 0.3 \cdot \min(c, 4) + 0.7 \cdot 1[\text{factual verb}] \tag{5.5}$$

where e is the number of named entities, 1[number] is 1 when the sentence contains a digit, c is the number of capitalised words and 1[factual verb] is 1 when it contains words such as "announced", "confirmed", "won" or "percent". The last two terms apply to English only. The eight highest-scoring sentences, kept in article order, are the claims. A short input such as a single headline becomes one claim.

5.6.2 Step 2: evidence retrieval

Every evidence passage is embedded once with BGE-M3 [31] and stored in a FAISS [32] inner-product index. Because the vectors are normalised, the inner product equals the cosine similarity

$$sim(q, d) = (q \cdot d) / (\|q\| \, \|d\|) \tag{5.6}$$

For each claim the three most similar passages are returned, and passages with similarity below 0.45 are dropped. BGE-M3 places English and Telugu text in the same space, so a Telugu claim finds English evidence. If a Google Fact Check Tools API key is configured, published fact-checks for the claim are added to the evidence.

5.6.3 Step 3: natural language inference

Each evidence passage e is paired with the claim c, the passage as premise and the claim as hypothesis, and the NLI model returns three probabilities: entailment, neutral and contradiction. English claims use DeBERTa-v3-large trained on MNLI, FEVER-NLI, ANLI, LingNLI and WANLI [30]; Telugu claims use the multilingual mDeBERTa-v3-base model. Over the claim's evidence set E the strongest support and the strongest contradiction are kept:

$$s^{+} = \max_{e \in E} p_{ent}(e, c), \quad s^{-} = \max_{e \in E} p_{con}(e, c) \tag{5.7}$$

5.6.4 Step 4: decision rule

With threshold $\tau = 0.5$, the claim's label is:

- **Contradiction** if $s^{-} \geq \tau$ and $s^{-} \geq s^{+}$, with confidence s^{-};
- **Support** if $s^{+} \geq \tau$, with confidence s^{+};
- **Unknown** otherwise, or when no evidence was found, with confidence $1 - \max(s^{+}, s^{-})$.

Contradiction is tested first. A single passage that clearly refutes a claim outweighs passages that only partly agree, which is how a human fact-checker treats a refutation. This rule is what makes the verification contradiction-aware. A system that ranks evidence by similarity alone would find the same passage for "the notes contain a GPS chip" and "the notes do not contain a GPS chip" and could not tell the two apart.

5.6.5 Step 5: aggregation

With n claims, labels y_i and confidences q_i, the article score is the average signed confidence

$$S = \frac{1}{n} \sum_i v_i, \quad \text{where } v_i = q_i \text{ for Support}, -q_i \text{ for Contradiction}, 0 \text{ for Unknown} \tag{5.8}$$

and the credibility score is $C = (S + 1) / 2$, between 0 and 1. The article is labelled Unverified when no claim is supported or contradicted, Likely Misleading when $S \leq -0.2$, Likely Credible when $S \geq 0.2$, and Mixed Evidence otherwise.

```
Algorithm: VerifyArticle(text, language)
  claims  <- ExtractClaims(text, language)           # at most 8
  results <- []
  for c in claims:
      E <- [p for p in FAISS.search(BGE_M3(c), k=3) if p.similarity >= 0.45]
      E <- E + FactCheckAPI(c)                         # optional
      if E is empty: results.add(c, Unknown, 0); continue
      for e in E: e.nli <- NLI(premise=e.text, hypothesis=c)
      s_plus  <- max(e.nli.entailment for e in E)
      s_minus <- max(e.nli.contradiction for e in E)
      if s_minus >= 0.5 and s_minus >= s_plus:
          results.add(c, Contradiction, s_minus)
      else if s_plus >= 0.5:
          results.add(c, Support, s_plus)
      else:
          results.add(c, Unknown, 1 - max(s_plus, s_minus))
  S <- mean(+q for Support, -q for Contradiction, 0 for Unknown)
  return results, credibility = (S + 1) / 2
```

5.7 Explainability Methods

5.7.1 LIME for text

LIME [21] explains one prediction f(x) by fitting a simple model g that behaves like f near x. It creates 200 copies of the article with random subsets of words removed, runs the classifier on each copy, and weights each copy by how similar it is to the original (the kernel π). It then solves

$$\xi(x) = \text{argmin over } g \text{ of } L(f, g, \pi_x) + \Omega(g) \tag{5.9}$$

where L measures how far g's outputs are from f's on the copies and Ω limits g to 12 words. The weight of each word in g is its explanation: positive weights push toward fake, negative toward real. The article is split on whitespace instead of an English tokenizer, so that Telugu words, whose vowel signs an English tokenizer would split off, stay whole.

5.7.2 Grad-CAM for images

Grad-CAM [22] uses the activations A of a chosen layer. For the predicted class c it averages the gradient of the class score over the spatial positions to get one weight per channel k, and combines the channels:

$$\alpha_k = (1/Z)\, \Sigma_{ij}\, \partial y^c / \partial A^k_{ij}, \quad L = ReLU(\Sigma_k\, \alpha_k\, A^k) \tag{5.10}$$

For CLIP the layer is the first layer normalisation of the last transformer block. Its 256 patch tokens are reshaped into a 16 by 16 grid before the formula is applied, and the map is upsampled to the image size and blended over it.

5.8 Fusion of Classifier and Evidence

When both the classifier and the claim verification have a result, the final fake risk is their average:

$$r = ½ (p_fake + (1 − C)) \quad (5.11)$$

The article is labelled Fake when r ≥ 0.5 and Real otherwise. If the claims are Unverified, the classifier alone decides, and if the classifier is not available the claims alone decide. The report always states which sources were used.

5.9 Training Setup

All models were trained in one notebook on a single NVIDIA T4 GPU (16 GB) in a free Kaggle session. The text models use the Hugging Face Trainer with the AdamW optimiser [41], a linear learning-rate schedule with 10% warm-up, weight decay 0.01, 16-bit mixed precision, and early stopping when validation F1 has not improved for two evaluations. The best checkpoint by validation F1 is kept. The times in Table 5.4 are the notebook's planning estimates for a T4; actual times vary with the session.

Table 5.4: Training hyperparameters

Model	Learning rate	Effective batch	Epochs	Max length	Estimated time on T4
DeBERTa-v3-large	1e-5	32 (8 × 4 accumulation)	2	256 tokens	1.5 to 2.5 hours
BERT-base	2e-5	32 (16 × 2)	3	256 tokens	30 to 60 minutes
IndicBERT v2	3e-5	32 (16 × 2)	4	256 tokens	about 20 minutes, plus 30 to 40 minutes of translation
CLIP linear head	1e-3	64	30	224 px	20 to 40 minutes, mostly feature extraction

5.10 Evaluation Metrics

With TP, TN, FP and FN the counts of true positives, true negatives, false positives and false negatives (fake, or AI-generated, is the positive class):

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (5.12)$$

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN) \quad (5.13)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall) \quad (5.14)$$

ROC-AUC is the area under the curve of true positive rate against false positive rate as the decision threshold moves from 0 to 1; it measures how well the model ranks fake items above real ones regardless of threshold. Claim verification has three classes, so it is measured with accuracy and macro-F1, the unweighted mean of the F1 scores of Support, Contradiction and Unknown, which gives the rare Contradiction class the same weight as the others.

CHAPTER 6
6. IMPLEMENTATION

This chapter describes how the design was turned into working code: the tools used, the structure of the project, the backend, the frontend, the training notebook and the launcher. Short excerpts of the real source code are included where they make a point clearer than prose.

6.1 Technologies Used

Table 6.1: Technology stack

Layer	Technology	Version	Role
Backend language	Python	3.10 to 3.12	All model and API code
Web framework	FastAPI + Uvicorn	0.11x	REST API, request validation, automatic API docs
Deep learning	PyTorch	2.x	Running and training models
Model library	Hugging Face Transformers	4.4x	Loading DeBERTa, IndicBERT, CLIP and NLI models
Embeddings	Sentence-Transformers	3.x	BGE-M3 evidence embeddings
Vector search	FAISS (CPU)	1.x	Nearest-passage search
Explanations	LIME, pytorch-grad-cam	0.2, 1.5	Word weights, heatmaps
NLP utilities	spaCy (en_core_web_sm)	3.x	Sentence splitting, named entities
Database	SQLite via SQLAlchemy	2.x	Check history
Frontend	React, React Router	18, 6	Single-page application
Build tool	Vite	6	Development server, /api proxy, production build
Styling	Tailwind CSS	4	Design tokens, light and dark themes
HTTP client	Axios	1.x	Calls from the browser to the API
Training	Kaggle / Google Colab notebook	T4 GPU	Fine-tuning and evaluation

6.2 Project Structure

```
truthlens-ai/
├── start.py, start.bat, start.sh     one-command launcher
```

```
├── backend/
│   ├── app/
│   │   ├── main.py                 FastAPI app, routers, error handlers
│   │   ├── config.py               settings read from .env
│   │   ├── database.py             SQLite table and save_check()
│   │   ├── schemas.py              request models (validation)
│   │   ├── routers/                text.py, image.py, batch.py, system.py
│   │   └── services/               language, text_classifier, text_explainer,
│   │                               claim_extractor, evidence, claim_verifier,
│   │                               image_detector, common
│   ├── models/                     trained models (text_en, text_te, image_detector)
│   ├── data/evidence/              evidence corpus (*.jsonl) and FAISS index
│   └── scripts/                    build_evidence_index.py, add_wikipedia_evidence.py
├── frontend/
│   └── src/
│       ├── pages/                  Home, TextCheck, ImageCheck, ClaimCheck,
│       │                           BatchCheck, History
│       ├── components/             Sidebar, VerdictScale, MarkedText, ClaimsReport,
...
│       ├── lib/                    verdict wording, samples, theme, page memory
│       └── api.js                  all calls to the backend
├── training/                       notebooks and training scripts
└── evaluation/                     eval_claims.py and curated claim set
```

6.3 Backend Implementation

6.3.1 Configuration

Every value that a student might want to change, such as model names, the evidence folder, the similarity threshold or the number of LIME samples, is a field of one `Settings` class. Pydantic reads it from `backend/.env`, so changing a model means editing one line of a text file. Model paths starting with `models/` are resolved inside the backend folder, and anything else is treated as a Hugging Face model name and downloaded on first use.

6.3.2 Lazy model loading

Loading all the models at start-up would take minutes and about 10 GB of memory. Instead, each model is created the first time it is needed. The `Lazy` wrapper uses a lock with a second check inside it, so two requests arriving together still load the model only once:

```
class Lazy(Generic[T]):
    """Builds an object the first time it is needed (thread-safe), then reuses it."""
    def get(self) -> T:
        if self._value is None:
            with self._lock:
                if self._value is None:
                    logger.info("Loading %s ...", self.name)
                    self._value = self._factory()
        return self._value
```

If a trained model folder is missing, the classifier raises `ModelNotReadyError`, which the API returns as HTTP 503 with a message that says which training script to run. The interface shows the model as "Not trained" instead of failing.

6.3.3 Language detection

```
TELUGU_CHARS = re.compile(r"[ఀ-౿]")
LETTERS = re.compile(r"[^\W\d_]", re.UNICODE)


def detect_language(text: str) -> str:
    telugu = len(TELUGU_CHARS.findall(text))
    letters = len(LETTERS.findall(text)) or 1
    return "te" if telugu / letters > 0.3 else "en"
```

6.3.4 Claim decision and fusion

The decision rule of Section 5.6 is a few lines of code. Keeping it this short makes it easy to explain in a viva and easy to test:

```
def _decide(evidence):
    if not evidence:
        return "Unknown", 0.0
    best_support = max(e["nli"]["entailment"] for e in evidence)
    best_contra = max(e["nli"]["contradiction"] for e in evidence)
    if best_contra >= DECISION_THRESHOLD and best_contra >= best_support:
        return "Contradiction", best_contra
    if best_support >= DECISION_THRESHOLD:
        return "Support", best_support
    return "Unknown", 1.0 - max(best_support, best_contra)


def combine(classification, verification):
    parts, based_on = [], []
    if classification:
        parts.append(classification["fake_probability"])
        based_on.append("classifier")
    if verification and verification["summary"]["verdict"] != "Unverified":
        parts.append(1.0 - verification["summary"]["credibility_score"])
        based_on.append("claim_evidence")
    if not parts:
        return None
    risk = sum(parts) / len(parts)
    return {"verdict": "Fake" if risk >= 0.5 else "Real",
            "fake_risk": round(risk, 4), "based_on": based_on}
```

6.3.5 Evidence index

The retriever reads every `*.jsonl` file in the evidence folder, embeds the passages with BGE-M3 and builds a FAISS `IndexFlatIP` index. It stores a fingerprint of the file names, sizes and modification times with the index, so the index is rebuilt automatically when a file is added or changed and reused otherwise. Adding evidence therefore needs no command: drop a file in the folder and restart.

6.3.6 Image detector and Grad-CAM

The detector class is the same in training and in the application, so the saved head loads without conversion. The CLIP embedding is normalised to unit length before the linear head:

```
class ClipDetector(nn.Module):
    def __init__(self, clip_name, num_labels):
        super().__init__()
        self.clip = CLIPVisionModelWithProjection.from_pretrained(clip_name)
```

```
        self.head = nn.Linear(self.clip.config.projection_dim, num_labels)

    def forward(self, pixel_values):
        emb = self.clip(pixel_values=pixel_values).image_embeds
        emb = emb / emb.norm(dim=-1, keepdim=True)
        return self.head(emb)
```

Grad-CAM expects a feature map shaped like an image, but a vision transformer produces a sequence of tokens. The reshape function drops the class token and arranges the 256 patch tokens into a 16 by 16 grid:

```
def reshape_transform(t):
    t = t[:, 1:, :]                          # drop CLS token
    return t.reshape(t.size(0), grid, grid, t.size(2)).permute(0, 3, 1, 2)

target_layer = self.model.clip.vision_model.encoder.layers[-1].layer_norm1
with GradCAM(model=self.model, target_layers=[target_layer],
             reshape_transform=reshape_transform) as cam:
    heatmap = cam(input_tensor=tensor, targets=[ClassifierOutputTarget(target_idx)])[0]
```

6.3.7 API and error handling

FastAPI validates every request against a Pydantic model before the handler runs. A text shorter than 10 characters or longer than 20,000 is rejected with HTTP 422, an oversized image with HTTP 413, and an unsupported file type with HTTP 415. FastAPI also generates interactive API documentation at `/docs`, which was used throughout development to test the endpoints by hand.

6.4 Frontend Implementation

6.4.1 Pages and routing

React Router maps each tool to a path (`/`, `/text`, `/image`, `/claims`, `/batch`, `/history`). The Start page sends its input to the right tool through router state, and the tool starts the check as soon as it opens, so the user goes from pasting text to reading the report with one click.

6.4.2 Verdict first, explanation second

LIME needs about 200 runs of the classifier, which on a laptop CPU takes much longer than the verdict itself. The text page therefore makes two requests. The first returns the verdict and the claim evidence and is shown at once; the second returns the word weights, and the highlights are drawn into the report already on screen:

```
// Step 1: verdict and claim evidence (fast)
data = await analyzeText({ text: input, language, explain: false,
                           verify_claims: verify });
setResult({ ...data, checkedText: input });
// Step 2: LIME highlights, filled into the report when ready
if (explain && data.classification) {
```

```
  const explanation = await explainText({ text: input, language: data.language });
  setResult((r) => r && r.checkedText === input
    ? { ...r, classification: { ...r.classification, explanation } } : r);
}
```

The check `r.checkedText === input` stops a slow explanation from an earlier check overwriting the report of a newer one.

6.4.3 Marked-up article

The `MarkedText` component splits the article the same way the backend's LIME explainer does, on whitespace, so each word lines up with its weight. Each weighted word is drawn with a highlighter-style background whose strength is proportional to the absolute weight; the marks appear one after another when the report loads. Claims are found in the text by their character positions, and every word inside a claim gets an underline whose style shows the claim's status. Hovering a claim in the evidence list sets an "active" index that lights up the matching sentence.

6.4.4 Theme and accessibility

All colours are design tokens defined once in CSS and redefined for dark mode, so the whole interface switches theme by changing one attribute on the page. A small script in `index.html` applies the saved theme before the first paint, so dark mode does not flash white. Every control can be reached with the keyboard and shows a visible focus ring, the verdict meters carry ARIA roles and values for screen readers, and animations are switched off when the operating system asks for reduced motion.

6.5 Training Notebook

The training notebook writes the training scripts to disk, installs packages, downloads the data, and runs each step only if its output does not exist yet. A disconnected session can therefore be resumed with "Run all". A `QUICK_TEST` switch trains every model on a few hundred examples in about 25 minutes, which checks the whole pipeline before the long run. The Kaggle version sets `QUICK_TEST = False`, saves all outputs to the notebook's output folder, and runs as a background commit, so the laptop can be closed while it trains.

The core of the text training script uses the Hugging Face `Trainer`:

```
training_args = TrainingArguments(
    learning_rate=args.lr, per_device_train_batch_size=args.batch_size,
    gradient_accumulation_steps=args.grad_accum, num_train_epochs=args.epochs,
    weight_decay=0.01, warmup_ratio=0.1,
    eval_strategy="epoch", save_strategy="epoch",
```

```
    load_best_model_at_end=True, metric_for_best_model="f1",
    fp16=use_fp16, seed=args.seed)
trainer = Trainer(model=model, args=training_args,
                  train_dataset=train_ds, eval_dataset=val_ds,
                  data_collator=DataCollatorWithPadding(tokenizer),
                  compute_metrics=compute_metrics,
                  callbacks=[EarlyStoppingCallback(early_stopping_patience=2)])
trainer.train()
```

After training, each script evaluates on the test set and writes `metrics.json`, a classification report, a confusion matrix image and a training curve. The last notebook cell gathers the results into one table and packages the three application models into `truthlens_models.zip`.

6.6 One-Command Launcher

Final-year projects are often hard to run on an examiner's computer because of missing packages or a wrong Python version. The launcher `start.py` (run by `start.bat` on Windows) removes these steps:

1. It checks that Python is version 3.10 to 3.12 and that Node.js is installed.
2. It creates the backend virtual environment if it does not exist.
3. It installs the Python packages, but only when the hash of `requirements.txt` has changed since the last install, so later starts take seconds.
4. It does the same for the frontend packages with `npm install`.
5. It checks that ports 8000 and 5173 are free, starts Uvicorn and Vite, waits for the health endpoint to answer, and opens the browser.
6. On Ctrl+C it stops both servers and their child processes, using `taskkill /T` on Windows.

```
def setup_backend():
    if not VENV_PYTHON.exists():
        run([sys.executable, "-m", "venv", VENV], ROOT,
            "Creating Python virtual environment")
    req, marker = BACKEND / "requirements.txt", VENV / ".requirements.sha256"
    wanted = file_hash(req)
    if marker.exists() and marker.read_text().strip() == wanted:
        say("Python packages already installed.")
    else:
        run([VENV_PYTHON, "-m", "pip", "install", "-r", req], BACKEND,
            "Installing Python packages")
        marker.write_text(wanted)
```

CHAPTER 7
7. TESTING

Testing checked two different things: that the software does what the requirements in Chapter 3 say, and that the models are accurate on data they have not seen. This chapter covers the first. The model evaluation is reported in Chapter 8.

7.1 Testing Strategy

Table 7.1: Levels of testing

Level	What was tested	How
Unit testing	Single functions: language detection, claim extraction, decision rule, aggregation, fusion	Small inputs with known answers (`evaluation/run_unit_tests.py`)
Integration testing	Each API endpoint with the real models loaded	Scripted HTTP requests to the running backend (`evaluation/run_test_cases.py`)
System testing	The whole application in the browser: input, report, history, themes, phone layout	Scripted browser sessions with Playwright (Chromium)
Validation testing	Wrong inputs: empty, too long, wrong file type, oversized files, broken CSV	Scripted requests that expect a specific error code
Model testing	Accuracy of every model on held-out data	Test splits and public benchmarks (Chapter 8)

7.2 Unit Testing

The rule-based parts of the system were tested with inputs whose correct output can be worked out by hand. They are run by `evaluation/run_unit_tests.py`, which needs no trained model.

Table 7.2: Unit test cases (actual results from evaluation/run_unit_tests.py)

ID	Function	Input	Expected	Actual	Result
UT01	detect_language	Hyderabad is the capital of Telangana.	en	en	Pass
UT02	detect_language	హైదరాబాద్ తెలంగాణ రాజధాని.	te	te	Pass

ID	Function	Input	Expected	Actual	Result
UT03	detect_language	5జీ టవర్లు 2020 నుంచి ఉన్నాయి (Telugu with digits and 5G)	te	te	Pass
UT04	extract_claims	Article with a question and a call to action	False	False	Pass
UT05	extract_claims	Same article	False	False	Pass
UT06	extract_claims	Same article (3 factual sentences)	3	3	Pass
UT07	extract_claims	Single headline with no factual cues	1	1	Pass
UT08	_decide	entailment 0.9, contradiction 0.05	('Support', 0.9)	('Support', 0.9)	Pass
UT09	_decide	entailment 0.6, contradiction 0.7	('Contradiction', 0.7)	('Contradiction', 0.7)	Pass
UT10	_decide	entailment 0.3, contradiction 0.2	('Unknown', 0.7)	('Unknown', 0.7)	Pass
UT11	_decide	no evidence	('Unknown', 0.0)	('Unknown', 0.0)	Pass
UT12	aggregate	Support 0.9 + Contradiction 0.8	('Mixed Evidence', 0.525)	('Mixed Evidence', 0.525)	Pass
UT13	aggregate	Contradiction 0.95 + Unknown	('Likely Misleading', 0.2625)	('Likely Misleading', 0.2625)	Pass
UT14	aggregate	only Unknown claims	Unverified	Unverified	Pass
UT15	combine	fake probability 0.8, credibility 0.4	('Fake', 0.7)	('Fake', 0.7)	Pass
UT16	combine	fake probability 0.3, claims Unverified	('Real', 0.3, ['classifier'])	('Real', 0.3, ['classifier'])	Pass

All 16 unit tests passed. The script imports the real backend functions, so a change to a rule that breaks its documented behaviour shows up as a failed row.

7.3 Integration Test Cases

The integration tests were run with `evaluation/run_test_cases.py` against the running application with all trained models installed, on the development laptop (Intel Core Ultra 7 255H, 16 logical processors and 11 GB of memory available to Ubuntu

under WSL 2 on Windows, no GPU). The "Actual result" column is copied from the script's output, not written by hand. The models had already been loaded by an earlier run, so the times show the cost of the check itself.

Table 7.3: Integration test cases (actual results from evaluation/run_test_cases.py)

ID	Module	Test	Input	Expected	Actual result	Status
TC 01	System	Health check reports model status	GET /api/health	HTTP 200 with status of every model	HTTP 200; models reported: text_en, text_te, image, nli_en, nli_multi, embeddings	Pass
TC 02	Validation	Reject text shorter than 10 characters	"hello"	HTTP 422, request rejected	HTTP 422	Pass
TC 03	Validation	Reject text longer than 20,000 characters	20,500 characters	HTTP 422, request rejected	HTTP 422	Pass
TC 04	Text check	English rumour is flagged fake	5G towers spread coronavirus (sample)	language en; overall verdict Fake	language=en, classifier=Fake (100.0%), overall=Fake (risk 0.87)	Pass
TC 05	Text check	English news report is judged real	Chandrayaan-3 landing report (sample)	language en; overall verdict Real	language=en, classifier=Real (100.0%), overall=Real (risk 0.00)	Pass
TC 06	Text check	Telugu text is detected and classified	Telugu sample (Telangana, 5G rumour)	language te; Telugu model gives a verdict	language=te, classifier=Fake (95.0%), claims={'Support': 2, 'Contradiction': 1, 'Unknown': 0}	Pass
TC 07	Text check	Claim checking can be switched off	verify_claims=false	verdict only, no claim results	language=en, verification=skipped	Pass
TC 08	Explanation	LIME returns word weights	5G rumour sample	a weight for each word, top words listed	63 tokens weighted; top words: deleted!, Share, across, The, the	Pass
TC 09	Claim check	Four claims labelled against evidence	Chandrayaan, GPS chip, UNESCO anthem, Hyderabad	Support, Contradiction, Contradiction, Support	labels=['Support', 'Contradiction', 'Contradiction', 'Support']; summary=Mixed Evidence	Pass
TC 10	Claim check	Claim with no evidence is Unknown	Polavaram claim (not in corpus)	Unknown; article Unverified	labels=['Unknown']; summary=Unverified	Pass

ID	Module	Test	Input	Expected	Actual result	Status
TC 11	Claim check	Telugu claim matched with English evidence	హైదరాబాద్ తెలంగాణ రాష్ట్ర రాజధాని.	Support, using the English Hyderabad passage	label=Support (98%); evidence: Hyderabad is the capital of the Indian state of Telangana.	Pass
TC 12	Image check	Real photograph is judged real	CIFAKE real image	Real, with Grad-CAM heatmap	label=AI-Generated / Manipulated (97.7%); heatmap=yes	Fail
TC 13	Image check	Generated image is judged AI-generated	CIFAKE generated image	AI-Generated / Manipulated, with Grad-CAM heatmap	label=AI-Generated / Manipulated (94.5%); heatmap=yes	Pass
TC 14	Image check	Non-image file is rejected	text file renamed .png	HTTP 400 with a message	HTTP 400: notes.png is not a valid image file.	Pass
TC 15	Image check	Image over 15 MB is rejected	16 MB file	HTTP 413 with a message	HTTP 413: big.jpg is larger than 15 MB.	Pass
TC 16	Batch	CSV of three texts is checked	CSV with text column, 3 rows	3 results with labels and a job id	3 rows; labels=['Fake', 'Real', 'Fake']; counts={'Fake': 2, 'Real': 1}	Pass
TC 17	Batch	CSV without a text column is rejected	CSV with column 'body'	HTTP 400 naming accepted columns	HTTP 400: CSV needs a column named one of: text, news, content, article, title, headline.	Pass
TC 18	Batch	ZIP of four images is checked	ZIP: 2 real, 2 generated	4 results with labels	4 images; labels=['Real', 'Real', 'AI-Generated / Manipulated', 'AI-Generated / Manipulated']	Pass
TC 19	Batch	Batch results download as CSV	job id from TC16	CSV file with header and 3 rows	HTTP 200; 4 lines; header: row,text,language,prediction,confidence,fake_probability	Pass
TC 20	Batch	Invalid job id is refused	job id '../etc'	HTTP 400 or 404, no file served	HTTP 404	Pass
TC 21	History	Checks are saved in history	GET /api/history	records of the checks above	40 records; kinds=['batch_image', 'batch_text', 'claims', 'image', 'text']	Pass
TC 22	History	History filter by type	kind=image	only image checks returned	16 records, all kind=image: True	Pass
TC 23	History	Deleting a missing record	DELETE /api/history/99999999	HTTP 404 Record not found	HTTP 404: Record not found.	Pass

22 of 23 integration tests passed. The one failure, TC12, is a model error rather than a software fault: the detector called a real CIFAKE photograph AI-generated with 97.7% confidence. The test was left as a failure instead of being rerun with a different image. To see how often this happens, all 12 sample images were checked: 9 of 12 were classified correctly (5 of 6 real photographs and 4 of 6 generated images; the two missed generated images were called Real with low confidence, 52% and 60%). Twelve images are too few to estimate accuracy, so the figure of 93.7% on 10,000 held-out images in Chapter 8 remains the measure of the detector. The sample images were taken from the public CIFAKE archive and may include images that were also in the training sample.

TC06 is worth a note as well. The Telugu sample contains two true statements and one rumour, and the claim check labelled them exactly so (2 supported, 1 contradicted), while the Telugu classifier called the whole text fake with 95% confidence. This is the behaviour the claim report is designed for: the reader sees which part of the message is false.

7.4 System Testing

The system tests drive a real Chromium browser through the application with Playwright, at a window size of 1440 by 900 pixels (375 by 812 for the phone test). In the first run three tests failed: ST03, ST04 and ST14. All three were faults in the test script, not in the application. ST03 hovered the first numbered list on the page, which turned out to be the list of influential words rather than the claim list. ST04 and ST14 reloaded the page between steps, which clears TruthLens's in-memory results by design. The three tests were corrected to hover the claim list and to move between pages with the navigation rail, as a user does, and were run again (`evaluation/run_ui_retest.py`). The table shows the results of the corrected tests.

Table 7.4: System test cases (actual results from evaluation/run_ui_tests.py)

ID	Scenario	Expected result	Actual result	Status
ST01	Start page sends text to the text check	Text page opens and the check starts	opened /text, verdict shown after 31 s	Pass
ST02	Verdict shown before highlights	Verdict first, highlights added later with a progress note	verdict on screen while highlights pending: True; 15 words marked 74 s later	Pass

ID	Scenario	Expected result	Actual result	Status
ST03	Hovering a claim highlights it in the article	Matching sentence lit up	hovering claim 2 lit up 27 words of that sentence in the article	Pass
ST04	Results kept when moving between pages	Report still shown on return	after moving to the image page and back with the rail, report and input still shown: True	Pass
ST05	Dark theme is remembered	Dark theme kept after reload	data-theme after reload = dark	Pass
ST06	Save as PDF uses a print layout	Navigation hidden when printing	navigation rail visible in print layout: False	Pass
ST07	Telugu article checked end to end	Telugu detected; verdict, highlights and claims shown	Telugu report shown with 14 marked words	Pass
ST08	Image report with heatmap slider	Slider wipes between image and heatmap	slider moved, heatmap style: clip-path: inset(0px 0px 0px 35%);	Pass
ST09	Start page sends an image to the image check	Image page opens and checks the image	image page opened and checked the dropped image	Pass
ST10	Verify claims page	Each claim listed with status and evidence	14 claims listed	Pass
ST11	Batch CSV check and download	Result CSV with one row per input	downloaded truthlens_text_a89604b3ee.csv with 3 rows	Pass
ST12	History filter	Only image checks shown	19 rows shown after filtering by Images	Pass
ST13	Phone layout	No horizontal scrolling at 375 px	page width 375 px at a 375 px screen	Pass
ST14	Keyboard-only use	Text box and Check button reachable with Tab	24 controls reached with Tab, including the text box and the Check button: True	Pass

7.5 Performance

Table 7.5: Measured response times on the development laptop (CPU only)

Operation	Input	Time
Health check	GET /api/health	under 0.1 s
Text check, verdict and claims	English rumour, 3 sentences (TC04)	13.7 s
Text check, verdict and claims	English news report, 2 sentences (TC05)	17.9 s

Operation	Input	Time
Text check, verdict and claims	Telugu text, 3 sentences (TC06)	10.4 s
Text check, verdict only	English report, claims off (TC07)	4.7 s
LIME explanation	English rumour, 63 words (TC08)	74.8 s
Claim verification	4 claims (TC09)	22.5 s
Image check with Grad-CAM	one image (TC12, TC13)	10 to 11 s
Batch text	3 rows, claims off (TC16)	6.4 s
Batch images	ZIP of 4 images (TC18)	10.5 s
Browser: verdict on screen	4-sentence article from the Start page (ST01)	31 s
Browser: highlights added	same article, after the verdict (ST02)	74 s more

The times come from the test runs above. The English DeBERTa-v3-large model dominates every text check: about 5 seconds for the verdict alone on a short text, and most of the rest goes to the NLI model checking each claim against three passages. LIME is the slowest step because it runs the classifier 200 times; on this CPU it took about 75 seconds for a 63-word text and grows with the length of the article. Splitting the response into a verdict request and an explanation request (Section 6.4) means the user waits for the verdict only, and the highlights appear in the open report when they are ready. With a CUDA GPU, which the system uses automatically when one is present, all of these steps are expected to be many times faster; this was not measured because the development laptop has no GPU.

7.6 Test Summary

Table 7.6: Test summary

Level	Test cases	Passed	Failed
Unit tests	16	16	0
Integration tests (API)	23	22	1
System tests (browser)	14	14	0
Total	53	52	1

Every functional requirement in Table 3.1 is covered by at least one passing test. The single failure is the image misclassification in TC12, which reflects the detector's measured error rate rather than a defect in the software. Model accuracy is reported separately in Chapter 8.

CHAPTER 8
8. RESULTS AND DISCUSSION

This chapter reports how well each part of TruthLens works. All numbers come from data the models never saw during training: the held-out test splits for the three classifiers, and public benchmarks plus a hand-written claim set for claim verification. Every figure and table can be regenerated from the notebook outputs and from `evaluation/eval_claims.py`.

8.1 English Text Classification

Table 8.1: Test results of the text and image classifiers

Model	Test set	Accuracy	Precision	Recall	F1	ROC-AUC
TF-IDF + logistic regression	WELFake (cleaned)	0.9615	n/a	n/a	0.9571	n/a
BERT-base (baseline)	WELFake, 12,727 articles	0.9884	0.9901	0.9844	0.9872	0.9993
DeBERTa-v3-large (proposed)	WELFake, 12,727 articles	0.9972	0.9979	0.9960	0.9970	0.9999
TF-IDF + logistic regression	WELFake-Telugu	0.8710	n/a	n/a	0.8748	n/a
IndicBERT v2	WELFake-Telugu, 2,000 articles	0.9305	0.9104	0.9550	0.9322	0.9832
CLIP ViT-L/14 + linear head	CIFAKE, 10,000 images	0.9371	0.9367	0.9376	0.9371	0.9839

Precision, recall and F1 are for the positive class (Fake, or AI-generated). The TF-IDF rows come from the bias check, which uses its own 80/20 split of 30,000 English or 10,000 Telugu rows, so they are a reference point rather than a matched comparison.

DeBERTa-v3-large made about 36 mistakes on 12,727 test articles, against about 148 for BERT-base, roughly a quarter as many. Of DeBERTa's errors, about 23 were fake articles it called real and about 12 were real articles it called fake. The model is slightly more likely to miss a fake article than to raise a false alarm, which is the safer direction for a tool that always shows its evidence next to the verdict.

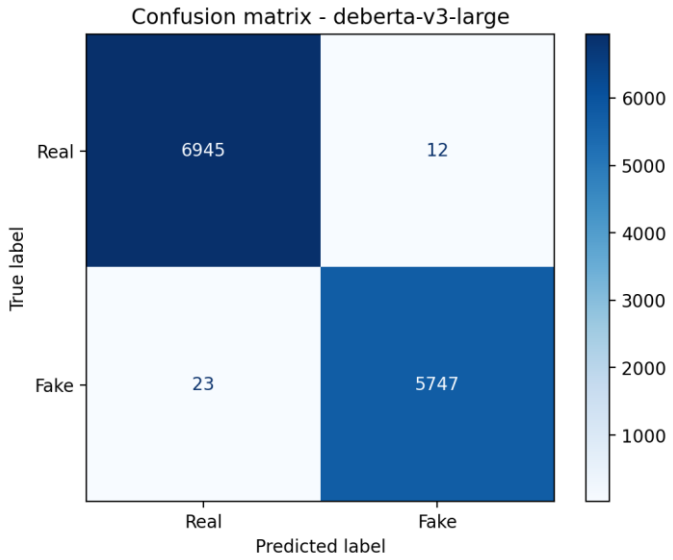


Figure 8.1: Confusion matrix of DeBERTa-v3-large on the English test set

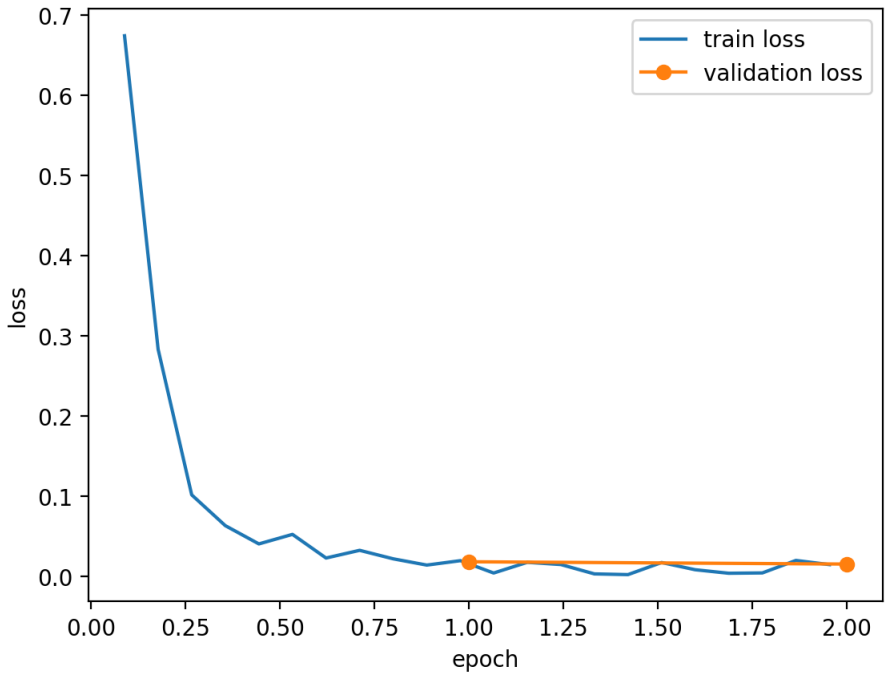


Figure 8.2: Training and validation loss of DeBERTa-v3-large

The training loss falls from 0.67 to below 0.05 within the first half epoch and stays near zero after that. The validation loss, measured at the end of each epoch, is about 0.02 after the first epoch and slightly lower after the second, so the model did not over-fit in the two epochs it was trained for.

The very high scores need to be read alongside the TF-IDF result. A model that only counts words reaches 96.2% on the same data, so the cleaned WELFake still contains strong style signals. DeBERTa's gain over TF-IDF, from 3.9% error to 0.3%, shows that it learns more than vocabulary. It does not show that the model would reach 99.7%

on fake news written in a different style, which is why TruthLens does not rely on the classifier alone.

8.2 Telugu Text Classification

IndicBERT v2 reached 93.1% accuracy and an F1 of 0.932 on 2,000 translated test articles, six points above the 87.1% of the TF-IDF model. The gap between the English and Telugu results has three likely causes:

1. **Less data.** The Telugu model was trained on 7,200 articles, the English model on 18,000 (the rest of each training part is held back for validation).
2. **Translation noise.** NLLB-200 sometimes drops or garbles names and numbers, and every article was cut to its first 120 words before translation, so the Telugu model sees less of each article.
3. **Smaller model.** IndicBERT v2 is a base-sized model, while DeBERTa-v3-large is a large one.

Table 8.2: Per-class results of the Telugu classifier

Class	Precision	Recall	F1	Test articles
Real	0.9527	0.9060	0.9288	1,000
Fake	0.9104	0.9550	0.9322	1,000

The Telugu model's errors lean the other way from the English model's. It catches 95.5% of fake articles but calls about 94 of the 1,000 real ones fake (real recall 90.6%). For a reader this means a Telugu "Likely fake" verdict deserves a look at the claim evidence before it is trusted.

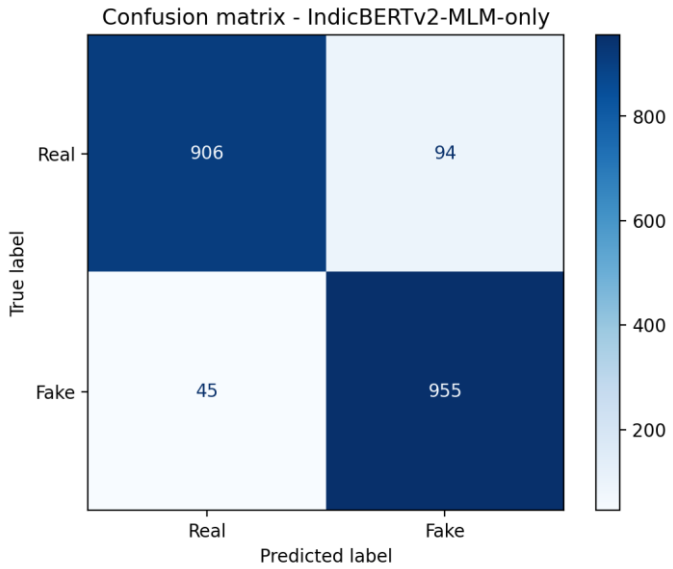


Figure 8.3: Confusion matrix of IndicBERT v2 on the Telugu test set

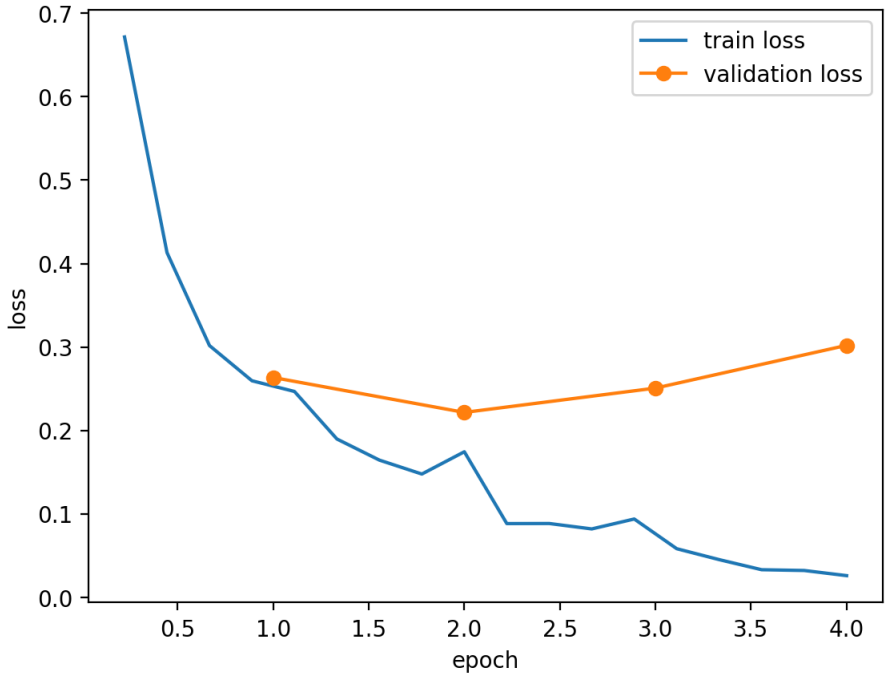


Figure 8.4: Training and validation loss of IndicBERT v2

The Telugu curve tells a different story from the English one. The validation loss is lowest after the second epoch (about 0.22) and rises after that to about 0.30 by the fourth, while the training loss keeps falling. The model starts to over-fit the 8,000 training articles after two epochs. Because the trainer keeps the checkpoint with the best validation F1 rather than the last one, the saved model is not the over-fitted final state, but a future run could stop after two or three epochs with no loss in quality.

8.3 AI-Generated Image Detection

The CLIP linear head reached 93.7% accuracy on 10,000 CIFAKE test images, with almost equal results for both classes: 93.7% recall on real photographs and 93.8% on generated images. Only 1,538 parameters were trained, and the whole training step after feature extraction took a few minutes, which shows how much of the work CLIP's pre-trained features already do.

Table 8.3: Per-class results of the image detector

Class	Precision	Recall	F1	Test images
Real	0.9375	0.9366	0.9371	5,000
AI-generated	0.9367	0.9376	0.9371	5,000

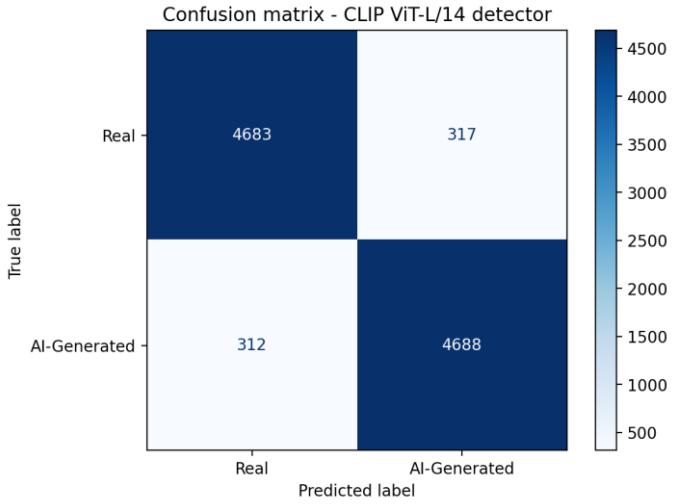


Figure 8.5: Confusion matrix of the CLIP image detector on CIFAKE

CIFAKE images are 32 by 32 pixels and are enlarged to 224 by 224 for CLIP, so most fine detail is lost before the model sees the image. The 93.7% is therefore a lower bound on what the approach can do with full-resolution images, but also a result for only one generator.

8.4 Claim Verification

Claim verification was evaluated in three ways. On FEVER and Climate-FEVER the gold evidence sentences are given with each claim, so the test measures the decision stage alone: given the evidence, does the system reach the right label? On the curated set the full pipeline runs, retrieval included, over the project's own evidence corpus. Every benchmark is balanced: FEVER has 300 claims of each label, Climate-FEVER 150 of each. The results are compared with a similarity-only baseline that labels a claim

Support whenever a passage with cosine similarity of at least 0.45 exists and Unknown otherwise, which is what a retrieval system without NLI can do.

Table 8.4: Claim verification results (threshold τ = 0.5)

Benchmark	Method	Accuracy	Macro-F1	F1 Support	F1 Contradiction	F1 Unknown	Claims
FEVER (dev)	Similarity only	0.400	0.292	0.539	0.000	0.336	900
FEVER (dev)	TruthLens	0.846	0.847	0.895	0.855	0.791	900
Climate-FEVER	Similarity only	0.340	0.181	0.503	0.000	0.039	450
Climate-FEVER	TruthLens	0.509	0.467	0.256	0.663	0.482	450
Curated set	Similarity only	0.525	0.487	0.628	0.000	0.833	40
Curated set	TruthLens	1.000	1.000	1.000	1.000	1.000	40

8.4.1 FEVER

On FEVER the decision rule labels 84.6% of the 900 claims correctly, with F1 between 0.79 and 0.90 for every class. The confusion matrix shows where the errors are. Supported and refuted claims are almost never confused with each other (2 supported claims called contradicted, 5 refuted claims called supported). Most errors involve Unknown: 37 supported and 45 refuted claims were marked Unknown because the NLI model was not confident enough, and 50 NotEnoughInfo claims were given a definite label.

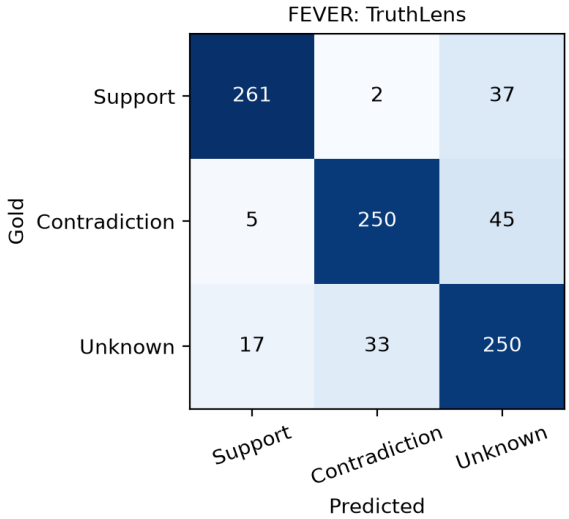


Figure 8.6: Confusion matrix of claim verification on FEVER

The similarity baseline shows why the NLI step is needed. It calls 287 of the 300 supported claims Support, but it also calls 251 of the 300 refuted claims Support, because a passage that refutes a claim is about the same subject and is just as similar to it. It finds no contradiction at all, since similarity has no notion of disagreement.

8.4.2 Climate-FEVER

Climate-FEVER is much harder. Its claims were collected from the web rather than written from Wikipedia, and its evidence sentences are related to the claim without always settling it. Accuracy falls to 50.9%. The errors are lopsided: TruthLens still catches 119 of the 150 refuted claims (recall 79%), but it labels only 23 of the 150 supported claims Support (recall 15%); most of the rest come back Unknown. When TruthLens does say Support, it is right 77% of the time.

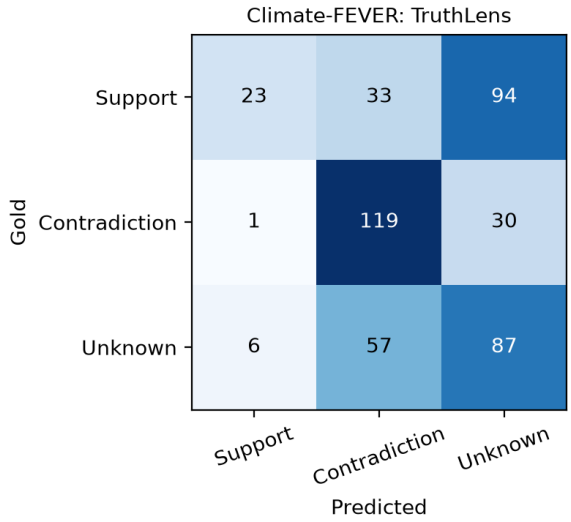


Figure 8.7: Confusion matrix of claim verification on Climate-FEVER

For a fact-checking aid this is the safer way to fail. A true claim marked "No evidence found" costs the reader a look at the evidence, while a false claim marked Supported would mislead them. The result also shows that the 84.6% on FEVER should not be read as the accuracy on real-world claims.

8.4.3 Telugu verification

The English NLI model was never trained on Telugu, so both NLI models were tested on 1,000 Telugu premise and hypothesis pairs from IndicXNLI [25], labelled entailment, neutral or contradiction.

Table 8.5: Three-class NLI on IndicXNLI Telugu (1,000 pairs)

NLI model	Accuracy	Macro-F1
DeBERTa-v3-large, English only	0.400	0.396
mDeBERTa-v3-base, multilingual (used for Telugu)	0.742	0.745

The English model, though larger and much stronger on English, is barely above the 33% of random guessing on Telugu. The multilingual model reaches 74.2%. This is why Telugu claims are sent to mDeBERTa, and it also shows that verification in Telugu is less reliable than in English.

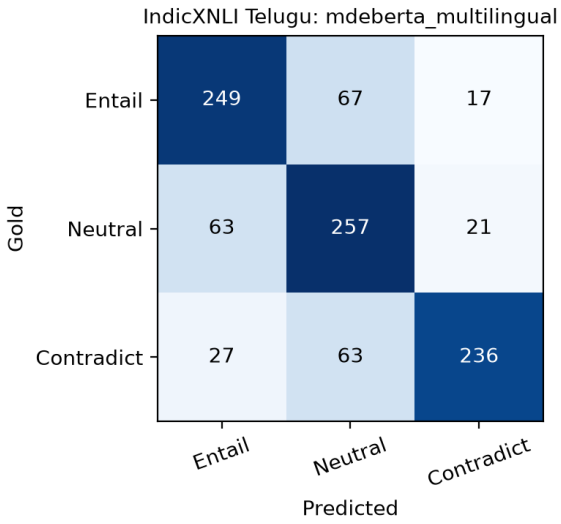


Figure 8.8: Confusion matrix of mDeBERTa-v3-base on IndicXNLI Telugu

8.4.4 Curated set (full pipeline)

On the 40 curated claims (30 English, 10 Telugu), the full pipeline labelled every claim correctly, including all 17 contradicted claims, while the similarity baseline marked all 17 contradicted claims as supported. Two examples from the predictions file show the difference:

Table 8.6: Two curated claims and their predictions

Claim	Gold label	TruthLens	Similarity only	Top evidence
India became independent from Britain in August 1947.	Support	Support	Support	India gained independence from British rule on 15 August 1947.

Claim	Gold label	TruthLens	Similarity only	Top evidence
India gained independence from British rule in 1950.	Contradiction	Contradiction	Support	India gained independence from British rule on 15 August 1947.

Both claims retrieve the same passage with high similarity. Only the NLI step notices that 1950 disagrees with 1947.

The 100% must be read carefully. The curated claims were written for this project against its own 40-passage corpus, so they test that retrieval, language routing and the decision rule work together. They are not a measure of accuracy on unseen claims, for which FEVER and Climate-FEVER are the better guide.

8.4.5 Choice of threshold

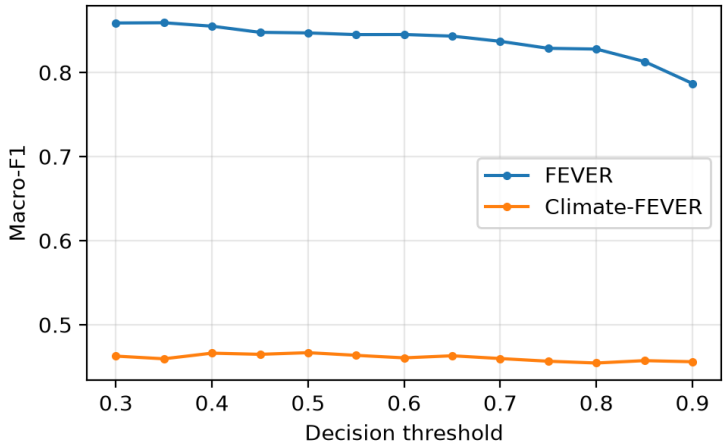


Figure 8.9: Macro-F1 of the decision rule as the threshold changes

The threshold τ decides how confident the NLI model must be before a claim is labelled Support or Contradiction. On FEVER the best macro-F1 is 0.859 at $\tau = 0.35$, slightly above the 0.847 at the default of 0.5, and it falls to 0.787 at $\tau = 0.9$ as more claims become Unknown. On Climate-FEVER the curve is flat between 0.455 and 0.467, so its weak Support recall comes from the NLI scores themselves and cannot be fixed by moving the threshold. The default of 0.5 was fixed before the evaluation and was kept, because a lower value would let weaker support through in the application.

8.5 Sample Outputs

The screenshots in this section were taken automatically with Playwright from the running application, with the trained models installed. Each shows a real result, not a mock-up.

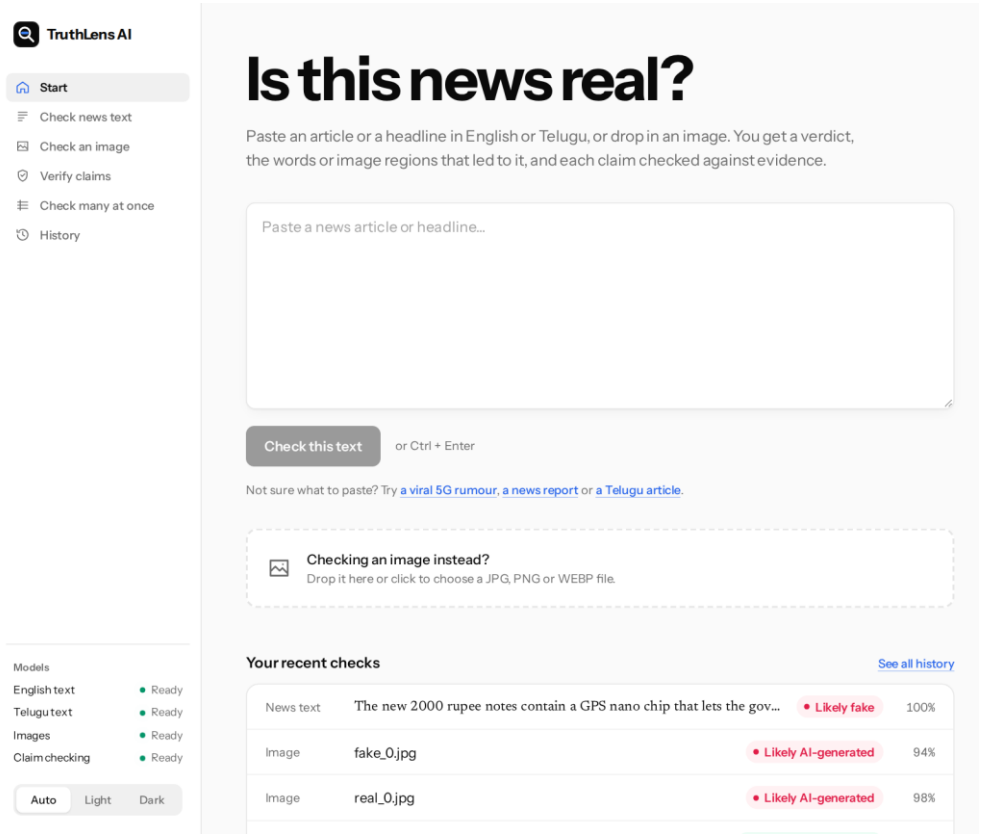


Figure 8.10: Start page

The Start page has a single text box and an image drop area. The left rail lists the tools and, at the bottom, the status of each model and the theme switch.

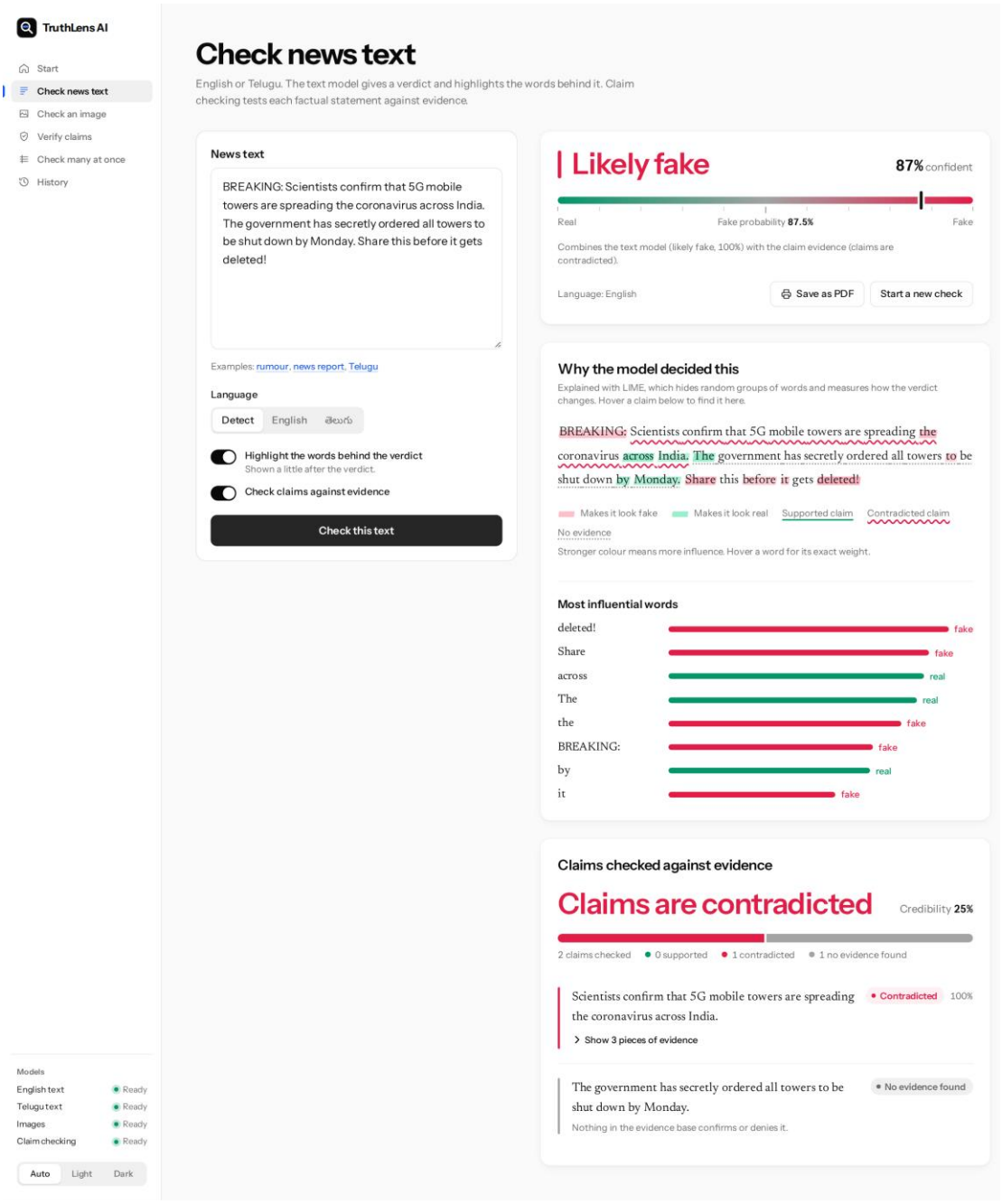


Figure 8.11: Report for an English rumour about 5G towers

For the 5G rumour the report gives "Likely fake" with 87% confidence. In the article, red marks show the words that pushed toward fake, among them "BREAKING", "Share" and "deleted!". The claim list below shows that the first claim, that 5G towers spread the coronavirus, is contradicted by the evidence passage, while the claim that the government ordered towers shut down has no evidence in the corpus and is marked "No evidence found". The overall verdict combines both sources.

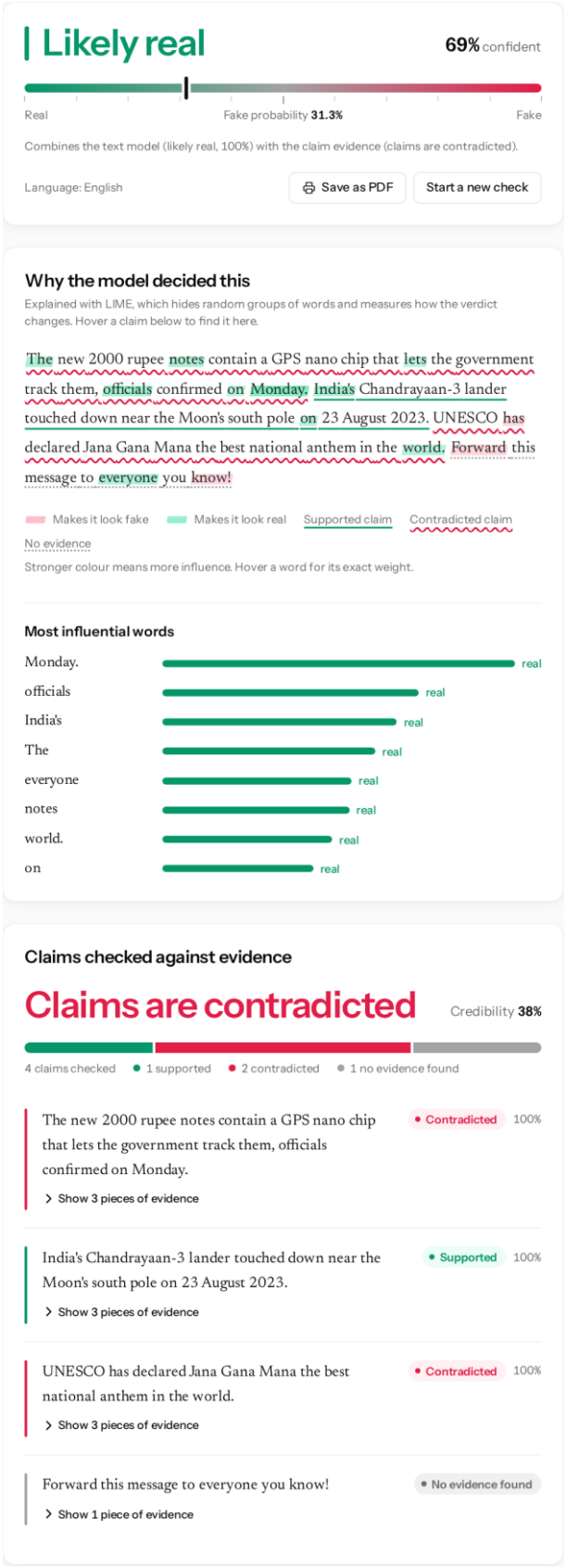


Figure 8.12: Hovering a claim lights up its sentence in the article

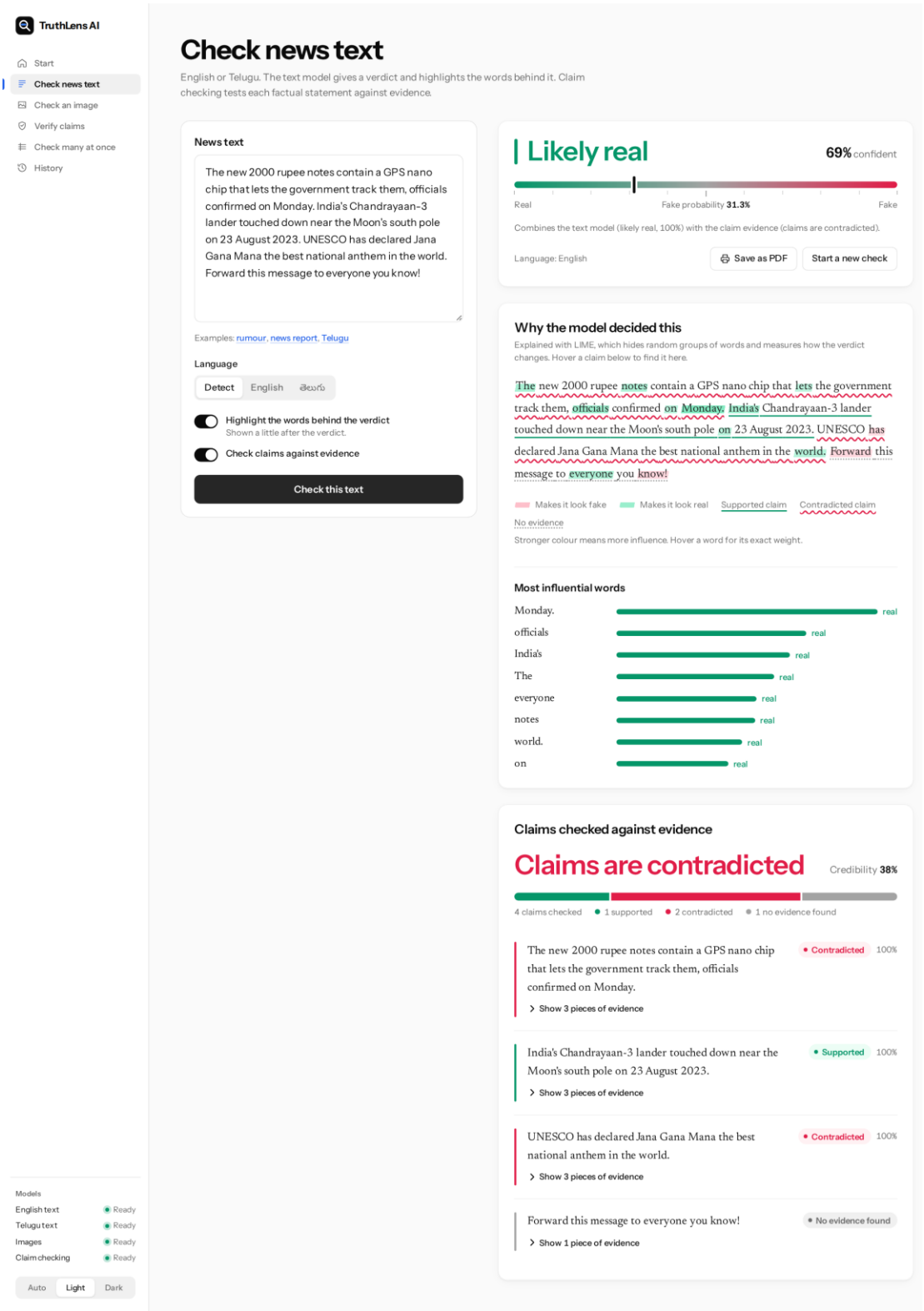


Figure 8.13: Report for an article that mixes true and false claims

The article in this report mixes one true claim (the Chandrayaan-3 landing) with two hoaxes (the GPS chip and the UNESCO anthem award). The claim report gets all three right: two contradicted, one supported, and the call to forward the message marked as having no evidence. The headline verdict, however, is "Likely real" at 69%, because

the classifier is confident that this calmly written text is real and the fused risk is the average of the two sources. This is the weakness discussed in Section 8.6, and it is the reason the claim list is shown on the same page as the verdict.

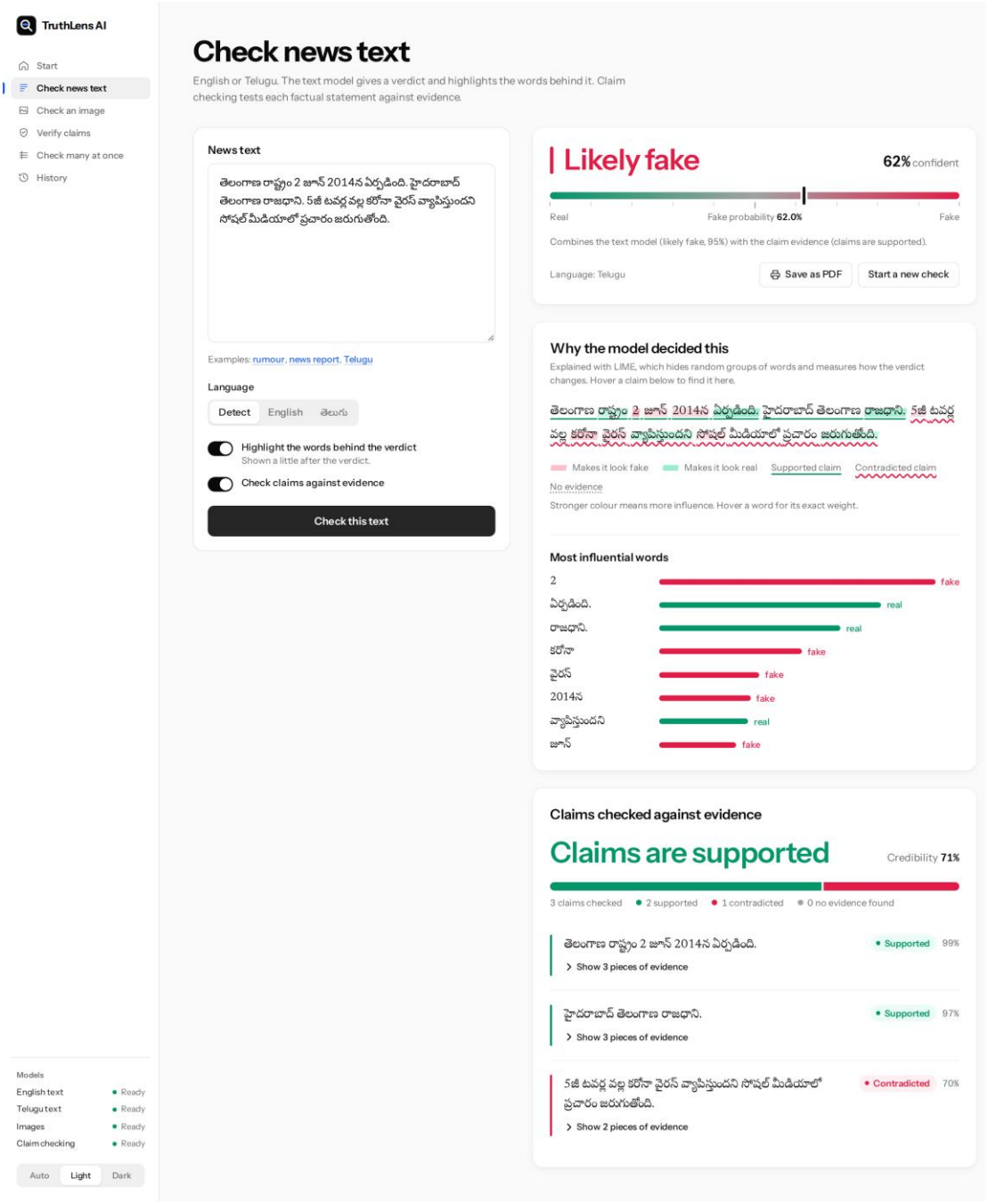


Figure 8.14: Report for a Telugu message

The Telugu message is detected as Telugu, sent to the IndicBERT v2 classifier and to the multilingual NLI model, and checked against the English evidence passages. Its two true statements about Telangana are supported and the 5G rumour is contradicted.

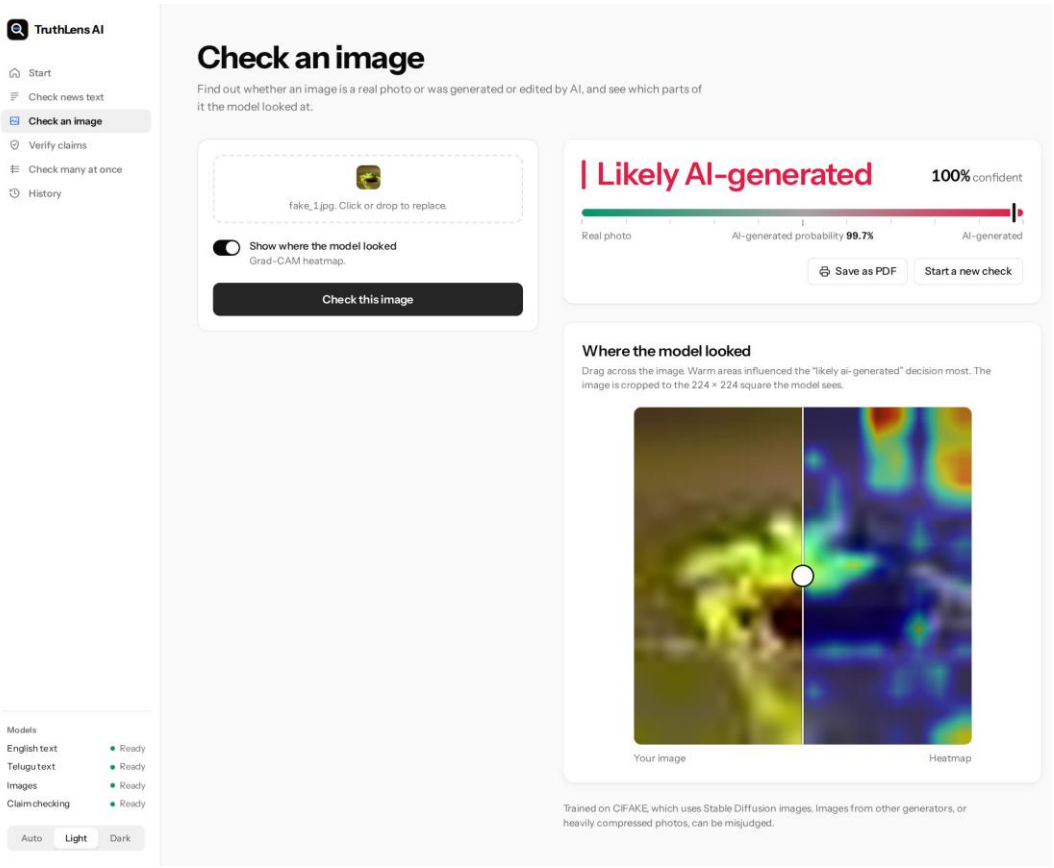


Figure 8.15: Image report for a generated image, with the Grad-CAM heatmap

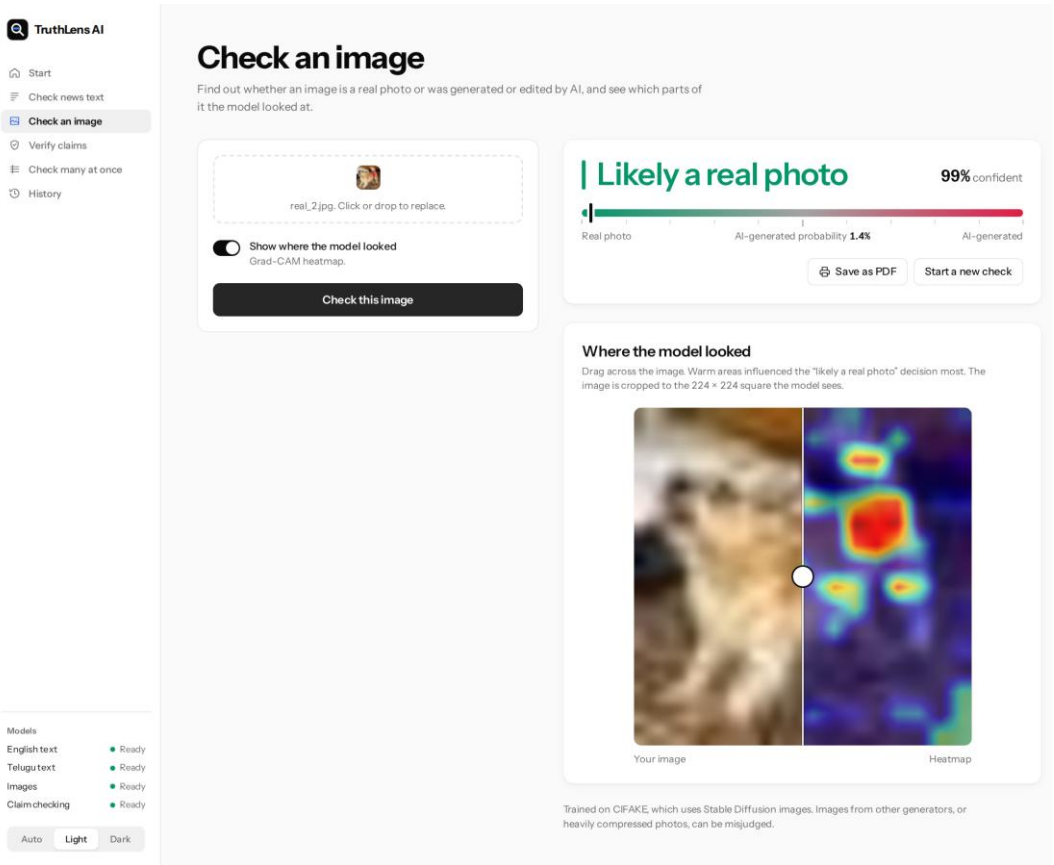


Figure 8.16: Image report for a real photograph

The image page shows the verdict on the same scale as the text page, and a slider that wipes between the image and its Grad-CAM heatmap. The CIFAKE images are 32 by 32 pixels, so they appear blurred when enlarged.

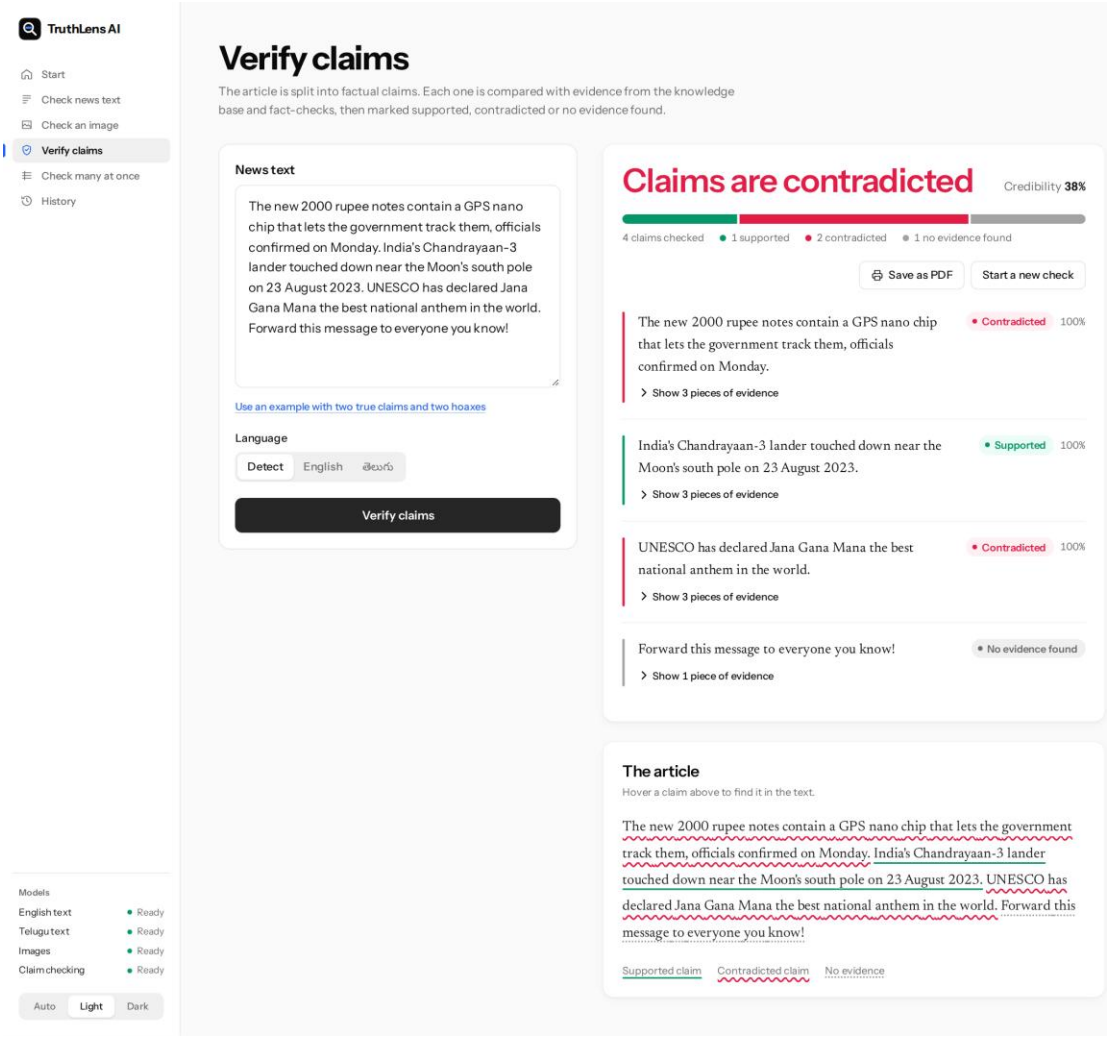


Figure 8.17: Verify claims page

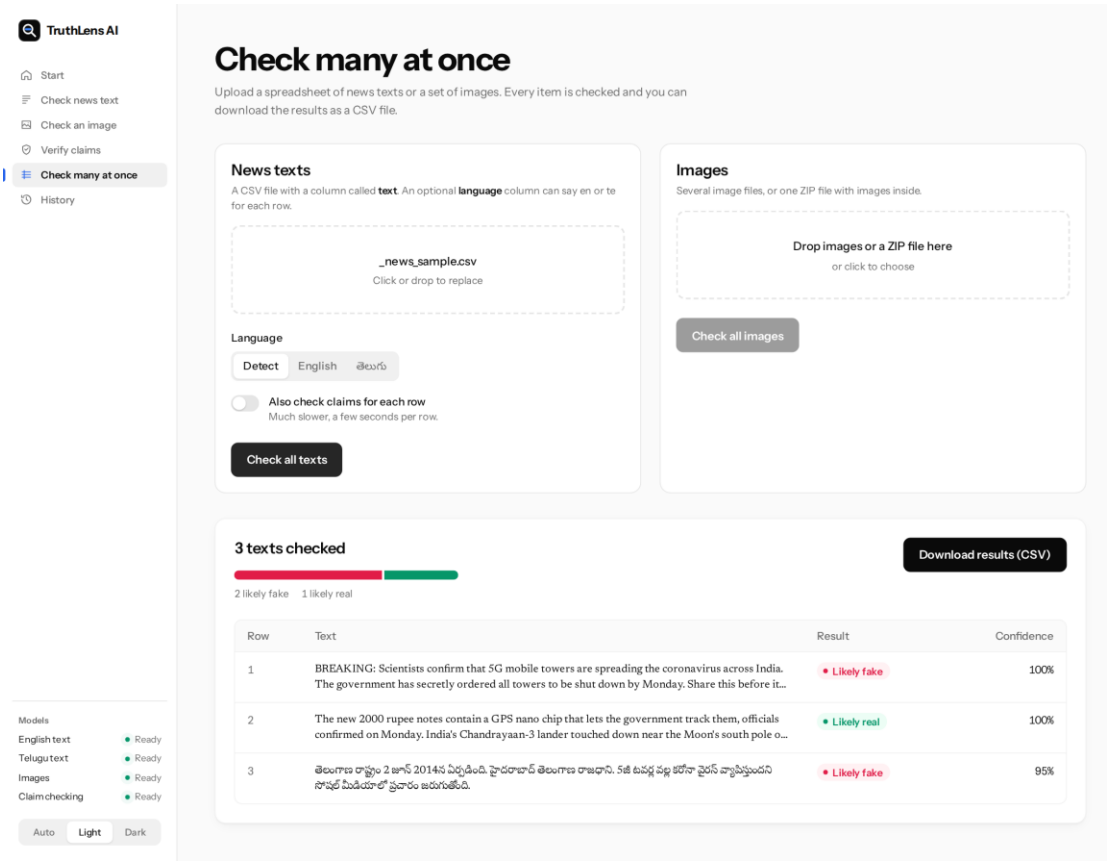


Figure 8.18: Batch check of three texts, with download

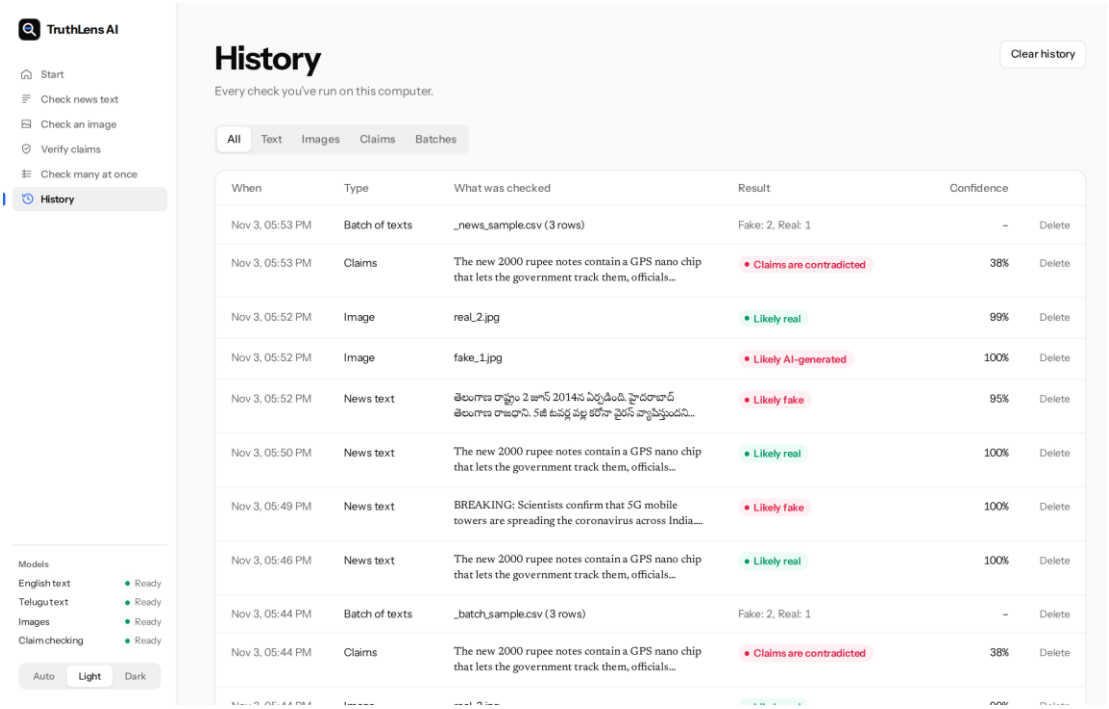


Figure 8.19: History of past checks

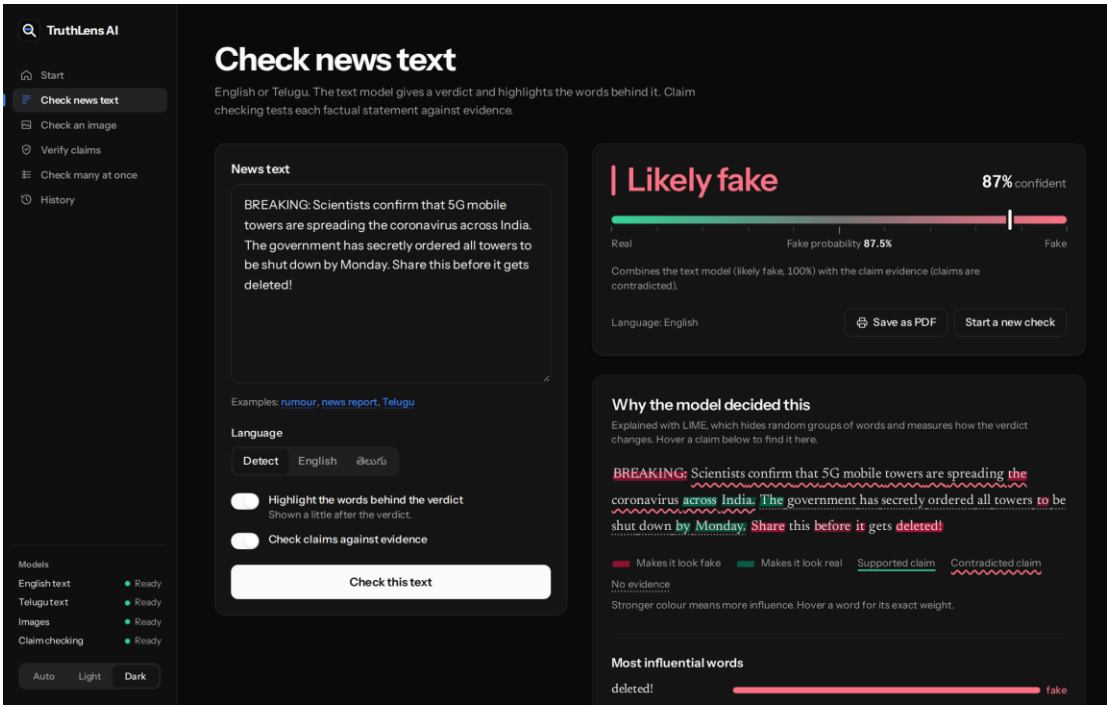


Figure 8.20: Report in the dark theme



Figure 8.21: Start page on a phone (375 px wide)

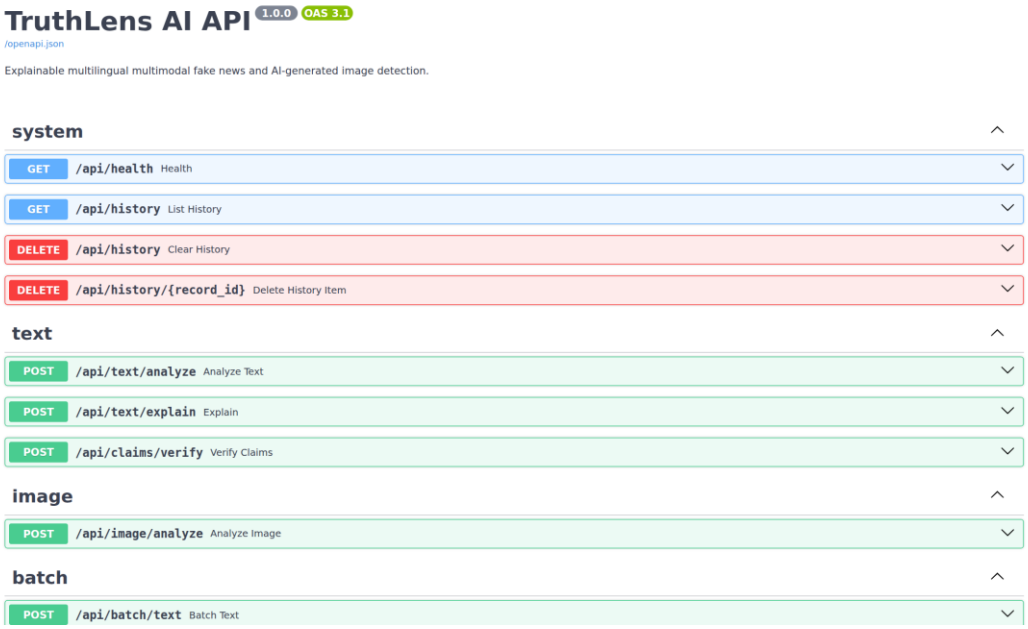


Figure 8.22: Interactive API documentation generated by FastAPI

8.6 Discussion

The results support three conclusions and set two warnings.

The classifiers work, but on data that is easier than real news. DeBERTa-v3-large reached 99.7% on English and IndicBERT v2 93.1% on Telugu. Both beat their baselines, and both beat a word-count model by a clear margin. But a word-count model reaches 96.2% on the same English data, which means WELFake's fake and real articles still differ in style even after cleaning. A classifier trained on it learns that style. That is useful when a new fake article is written like the old ones, and of little use when it is written carefully.

Explicit contradiction detection is the most important design decision. The similarity baseline failed to find a single refuted claim on any benchmark, while TruthLens found 83% of them on FEVER and 79% on Climate-FEVER. A retrieval system that ranks evidence by similarity tells the reader what the evidence is about, not whether it agrees. Testing for contradiction first, with an NLI model, is what turns retrieved passages into a verdict.

Claim verification is weakest on real-world support. On Climate-FEVER the system rarely confirms a true claim. It prefers "No evidence found" to a wrong confirmation, which suits a tool meant to make readers look at evidence, but it means a true article with unusual phrasing will often get few Supported claims. Better NLI models trained

on real-world evidence, or several evidence passages combined before the decision, are the most promising fixes.

A worked example shows both strengths and a weakness. Given the single sentence "The new 2000 rupee notes contain a GPS nano chip that lets the government track them, officials confirmed on Monday.", the English classifier answered Real with 99.98% confidence: the sentence is calm, specific and attributed to officials, which is how real reports in WELFake are written. The claim check retrieved the passage stating that the notes contain no chip, and the NLI model scored it as a contradiction with probability 1.00. The fused fake risk was 0.5001, so the final verdict was Fake, but only just. The example shows why the evidence stage matters, and it also shows the limit of simple averaging: a confident classifier can almost cancel a clear contradiction. Giving a strong contradiction more weight than the classifier, rather than averaging the two, is an obvious improvement for future work.

The first warning concerns the evidence corpus. Every claim result in the application depends on what the corpus contains. With 40 passages, most claims in an arbitrary article will be Unknown, and the fused verdict will then rest on the classifier alone. The method is sound; the coverage is small.

The second concerns Telugu. The Telugu classifier was trained and tested on translated articles about United States news, and the Telugu NLI model reaches 74.2%. The Telugu results in this chapter are therefore a measure of the method, not a promise about performance on native Telugu misinformation.

Table 8.7: Summary of results

Component	Test data	Main result	Compared with
English classifier (DeBERTa-v3-large)	WELFake, 12,727 articles	99.7% accuracy, F1 0.997	BERT-base 98.8%, TF-IDF 96.2%
Telugu classifier (IndicBERT v2)	WELFake-Telugu, 2,000 articles	93.1% accuracy, F1 0.932	TF-IDF 87.1%
Image detector (CLIP + linear head)	CIFAKE, 10,000 images	93.7% accuracy, AUC 0.984	n/a
Claim verification	FEVER, 900 claims	84.6% accuracy, macro-F1 0.847	Similarity only 40.0%
Claim verification	Climate-FEVER, 450 claims	50.9% accuracy, macro-F1 0.467	Similarity only 34.0%

Component	Test data	Main result	Compared with
Telugu NLI (mDeBERTa)	IndicXNLI, 1,000 pairs	74.2% accuracy	English NLI model 40.0%
Full pipeline	Curated set, 40 claims	40 of 40 correct	Similarity only 21 of 40

CHAPTER 9
9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project set out to build a fake news checker that a reader in Andhra Pradesh or Telangana could actually use: one that works in Telugu as well as English, checks images as well as text, and shows its reasons instead of asking to be trusted. TruthLens AI meets these goals in a working application that installs with one command and runs on a laptop.

The English classifier, DeBERTa-v3-large fine-tuned on cleaned WELFake, reached 99.7% accuracy on 12,727 held-out articles, against 98.8% for a BERT-base model trained the same way and 96.2% for a TF-IDF model. The Telugu classifier, IndicBERT v2 fine-tuned on a translated set built in this project, reached 93.1%, well above the 87.1% that word counts alone achieve on the same data. The image detector, a linear head on frozen CLIP ViT-L/14 features, reached 93.7% on CIFAKE.

Two findings from the data work matter as much as the scores. The WELFake labels are the reverse of their documentation, and the dataset's publisher names and credits let a model learn the source rather than the content; both had to be fixed before any number was meaningful. The public Telugu dataset turned out to be separable at 99.8% by a word-count model because its classes come from different kinds of source. Reporting a high accuracy on it would have been easy and misleading. The project instead built a translated set and measured its remaining bias openly.

The claim verification stage is the part of TruthLens that does not depend on writing style. It splits an article into claims, retrieves evidence with multilingual embeddings, and uses NLI with a contradiction-first decision rule to label each claim Supported, Contradicted or Unknown. On FEVER it reached 84.6% accuracy and 0.847 macro-F1, while a system that labels claims by similarity alone reached 40.0% and could not detect a single contradiction. This comparison is the main evidence for the project's central design choice. On the harder Climate-FEVER set, made of real claims from the web, accuracy fell to 50.9%: the system still caught 79% of the false claims but marked most true claims as having no clear evidence. For Telugu claims, the multilingual NLI model reached 74.2% on IndicXNLI, against 40.0% for the English model.

The interface puts all of this in one report: the verdict in plain words, the influential words marked in the article, each claim underlined by its status, and the evidence printed beside it.

9.2 Limitations

- **Evidence coverage.** Claim verification can only decide claims for which the corpus holds a relevant passage. The shipped corpus has 40 passages, so most claims in a random article will be Unknown. The verification method is sound; the evidence base is small.
- **Translated Telugu data.** The Telugu model was trained on machine-translated United States news. It has not been tested on native Telugu news, whose topics, names and style differ.
- **Remaining source bias.** Even after cleaning, a TF-IDF model scores 96.2% on WELFake, so part of the classifiers' accuracy still comes from the style of the sources. The very high English score should be read with this in mind.
- **Low-resolution images.** CIFAKE images are 32 by 32 pixels from one generator. The detector has not been tested on full-resolution images or on newer generators such as Midjourney or DALL-E 3.
- **Speed on CPU.** The English classifier and its LIME explanation are slow on a laptop CPU. The interface shows the verdict first to hide this, but highlights for a long article can take several minutes.
- **Real-world claims.** On Climate-FEVER only 15% of supported claims were recognised as supported. The NLI model needs close entailment, which evidence about real-world claims often does not give.
- **Telugu verification.** The multilingual NLI model reached 74.2% on Telugu NLI pairs, so Telugu claim labels are less reliable than English ones.
- **Two languages.** Hindi, Tamil and other Indian languages are not supported by the classifiers.

9.3 Future Work

1. **Larger evidence base.** Load Wikipedia in English and Telugu, the PIB Fact Check archive and the Google Fact Check Tools results into the index, and refresh it on a schedule so that recent claims can be checked.

2. **Native Telugu data.** Collect and label Telugu articles from fact-checking sites and regional news, with both classes drawn from comparable sources, and use it to test and retrain the Telugu model.
3. **More languages.** IndicBERT v2, BGE-M3 and the multilingual NLI model already cover Hindi, Tamil, Kannada and other languages, so adding a language needs mainly training data and a script check in the language detector.
4. **Modern image detection.** Train the CLIP head on full-resolution images from several current generators, and add checks for edited photographs as well as generated ones.
5. **Image and text together.** Check whether an image matches the claim in its caption, using CLIP's shared image and text space, since many false posts pair a real photo with a false description.
6. **Faster explanations.** Replace LIME with a gradient-based method such as integrated gradients, which needs one or two model passes instead of 200, or distil the English model into a smaller one for CPU use.
7. **Weighted fusion.** Let a strong contradiction outweigh the classifier instead of averaging the two, since a calm, well-written false claim can receive a confident Real from the classifier (Section 8.6).
8. **Browser extension and messaging bot.** Let readers check a message from inside WhatsApp or a news page without copying it into the web application.

CHAPTER 10
10. INSTALLATION AND USER GUIDE

This chapter explains how to install TruthLens AI on a new computer, how to install the trained models, and how to use each page of the application.

10.1 Prerequisites

Install these once:

1. **Python 3.10, 3.11 or 3.12** from python.org. On Windows, tick "Add python.exe to PATH" on the first screen of the installer. Python 3.13 is not yet supported by all the machine learning packages.
2. **Node.js 18 or later** (the LTS version) from nodejs.org.
3. About **15 GB of free disk space** for the Python packages and models.
4. An internet connection for the first run, which downloads the packages and the pre-trained models.

10.2 Installing and Starting

Windows. Open the `truthlens-ai` folder and double-click `start.bat`.

Linux or macOS. Open a terminal in the `truthlens-ai` folder and run:

```
python3 start.py
```

The first run creates the Python environment and installs the packages, which takes 5 to 15 minutes. It then starts the backend at http://127.0.0.1:8000 and the frontend at http://localhost:5173, and opens the browser. Later runs skip the installation and start in a few seconds. Press Ctrl+C in the terminal window to stop both servers.

Table 10.1: Launcher options

Command	Effect
python start.py	Install what is missing, start both servers and open the browser
python start.py --setup	Only install; do not start
python start.py --no-browser	Start without opening the browser

10.3 Installing the Trained Models

The training notebook produces `truthlens_models.zip`. Extract it into `backend/models/` so that the folder contains:

```
backend/models/text_en/          English model (DeBERTa-v3-large)
backend/models/text_te/          Telugu model (IndicBERT v2)
backend/models/image_detector/   Image detector (config.json + head.pt)
```

No restart is needed: the application loads a model the first time it is used. The NLI models and the BGE-M3 embedding model are downloaded automatically from Hugging Face on the first claim check, about 3 GB in total.

10.4 Configuration

All settings are in `backend/.env`, which the launcher creates from `.env.example` on the first run.

Table 10.2: Main configuration settings

Setting	Default	Meaning
TEXT_EN_MODEL, TEXT_TE_MODEL, IMAGE_MODEL	models/...	Folders of the trained models
NLI_EN_MODEL, NLI_MULTI_MODEL	MoritzLaurer DeBERTa models	NLI models for English and Telugu claims
EMBED_MODEL	BAAI/bge-m3	Evidence embedding model
EVIDENCE_DIR	data/evidence	Folder of evidence JSONL files
GOOGLE_FACTCHECK_API_KEY	empty	Optional free key to add published fact-checks
LIME_NUM_SAMPLES	200	Perturbed copies per text explanation; lower is faster
MAX_CLAIMS	8	Most claims taken from one article
EVIDENCE_TOP_K, EVIDENCE_MIN_SIMILARITY	3, 0.45	Passages per claim and the similarity cut-off
MAX_BATCH_ROWS	500	Most rows in one batch CSV
DEVICE	auto	cpu, cuda, or auto to use a GPU when present

10.5 Adding Evidence

To let the system check more claims, add passages to the evidence folder. Create a file such as `backend/data/evidence/my_facts.jsonl` with one passage per line (wrapped here to fit the page):

```
{"text": "ISRO's Chandrayaan-3 lander touched down near the Moon's south pole
          on 23 August 2023.", "source": "ISRO", "url": "https://www.isro.gov.in"}
```

Restart the backend. The index is rebuilt automatically because the folder changed. To add whole Wikipedia articles, run:

```
cd backend
.venv/Scripts/python scripts/add_wikipedia_evidence.py "Chandrayaan-3" "Polavaram
Project"
```

On Linux or macOS the Python path is `.venv/bin/python`. The script splits each article into passages and saves them as a new evidence file.

10.6 Using the Application

10.6.1 Start page

Paste a news article or headline into the text box and press the check button (or Ctrl+Enter), or drop an image into the image area. The application opens the right tool and starts the check.

10.6.2 Checking news text

1. Open "Check news text" from the left rail.
2. Paste the article, or press one of the example links under the box (rumour, news report or Telugu) to load a sample.
3. Leave the language on "Detect", or choose English or Telugu.
4. Leave "Highlight the words behind the verdict" and "Check claims against evidence" on for the full report, or switch them off for a faster verdict.
5. Press "Check this text" (or Ctrl+Enter).

The report shows the verdict and a fake-risk scale first. The highlighted article follows a little later: red marks are words that pushed the verdict toward fake, green marks toward real, and stronger colour means more influence. Each claim is underlined, solid for supported, wavy for contradicted and dotted for no evidence. The claim list below

shows each claim's status, the evidence passage, its source link and the NLI scores; hovering a claim lights up its sentence in the article.

10.6.3 Checking an image

Open "Check an image", drop or choose a JPG, PNG or WEBP file of up to 15 MB, and press "Check this image". The report shows the verdict, the confidence scale and the image with a slider. Drag the slider to compare the original with the Grad-CAM heatmap: warm colours mark the regions that drove the decision.

10.6.4 Verifying claims only

"Verify claims" runs claim extraction and evidence checking without the classifier. It is useful for a single statement, such as a forwarded message, when the reader only wants to know whether the evidence supports it.

10.6.5 Checking many at once

Open "Check many at once". For text, upload a CSV file with a column named `text` (and, optionally, a column named `language`). For images, choose several image files or one ZIP archive. After the check, the page shows a summary of the results and a table of rows; press "Download results (CSV)" to save them.

10.6.6 History and saving reports

Every check is listed on the History page with its time, type, input and verdict. Use the filter to show one type of check, the delete button to remove one entry, or "Clear history" to remove everything. On any report page, "Save as PDF" opens the browser's print dialog with a print layout that hides the navigation.

10.6.7 Light and dark theme

The switch at the bottom of the left rail offers Auto, Light and Dark. Auto follows the operating system's setting. The choice is remembered for the next visit.

10.7 Troubleshooting

Table 10.3: Common problems and solutions

Problem	Solution
"Python 3.10 to 3.12 is required"	Install Python 3.12 and run the launcher again
"npm was not found"	Install Node.js LTS and open a new terminal
"Port 8000 (or 5173) is already in use"	Close the other TruthLens window, or restart the computer

Problem	Solution
Package installation fails	Check the internet connection and run the launcher again; it resumes
A model shows "Not trained"	Extract truthlens_models.zip into backend/models
The first claim check is slow	The NLI and embedding models are downloading; later checks are faster
Highlights take minutes	Lower LIME_NUM_SAMPLES in backend/.env, or switch the highlighting off
Need a clean reinstall	Delete backend/.venv and frontend/node_modules, then run the launcher

REFERENCES

[1] S. Vosoughi, D. Roy, and S. Aral, "The spread of true and false news online," *Science*, vol. 359, no. 6380, pp. 1146–1151, 2018.

[2] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.

[3] W. Y. Wang, "'Liar, liar pants on fire': A new benchmark dataset for fake news detection," in *Proc. 55th Annual Meeting of the ACL*, 2017, pp. 422–426.

[4] P. K. Verma, P. Agrawal, I. Amorim, and R. Prodan, "WELFake: Word embedding over linguistic features for fake news detection," *IEEE Trans. Computational Social Systems*, vol. 8, no. 4, pp. 881–893, 2021.

[5] R. K. Kaliyar, A. Goswami, and P. Narang, "FakeBERT: Fake news detection in social media with a BERT-based deep learning approach," *Multimedia Tools and Applications*, vol. 80, pp. 11765–11788, 2021.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[7] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proc. ICLR*, 2021.

[8] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. ICLR*, 2023.

[9] M. Z. Hossain, M. A. Rahman, M. S. Islam, and S. Kar, "BanFakeNews: A dataset for detecting fake news in Bangla," in *Proc. 12th Language Resources and Evaluation Conference (LREC)*, 2020.

[10] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, 2020.

[11] S. Doddapaneni et al., "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proc. 61st Annual Meeting of the ACL*, 2023.

[12] NLLB Team et al., "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.

[13] "Telugu fake news dataset," Zenodo, 2024, doi: 10.5281/zenodo.11408513.

[14] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. 38th ICML*, 2021, pp. 8748–8763.

[15] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[16] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "CNN-generated images are surprisingly easy to spot... for now," in *Proc. IEEE/CVF CVPR*, 2020, pp. 8695–8704.

[17] U. Ojha, Y. Li, and Y. J. Lee, "Towards universal fake image detectors that generalize across generative models," in *Proc. IEEE/CVF CVPR*, 2023, pp. 24480–24489.

[18] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, "On the detection of synthetic images generated by diffusion models," in *Proc. IEEE ICASSP*, 2023.

[19] J. J. Bird and A. Lotfi, "CIFAKE: Image classification and explainable identification of AI-generated synthetic images," *IEEE Access*, vol. 12, pp. 15642–15650, 2024.

[20] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF CVPR*, 2022, pp. 10684–10695.

[21] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 1135–1144.

[22] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[23] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proc. NAACL-HLT*, 2018, pp. 809–819.

[24] T. Diggelmann, J. Boyd-Graber, J. Bulian, M. Ciaramita, and M. Leippold, "CLIMATE-FEVER: A dataset for verification of real-world climate claims," arXiv:2012.00614, 2020.

[25] D. Aggarwal, V. Gupta, and A. Kunchukuttan, "IndicXNLI: Evaluating multilingual inference for Indian languages," in *Proc. EMNLP*, 2022.

[26] Z. Guo, M. Schlichtkrull, and A. Vlachos, "A survey on automated fact-checking," *Trans. Association for Computational Linguistics*, vol. 10, pp. 178–206, 2022.

[27] N. Hassan, F. Arslan, C. Li, and M. Tremayne, "Toward automated fact-checking: Detecting check-worthy factual claims by ClaimBuster," in *Proc. 23rd ACM SIGKDD*, 2017, pp. 1803–1812.

[28] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. NAACL-HLT*, 2018, pp. 1112–1122.

[29] Y. Nie, A. Williams, E. Dinan, M. Bansal, J. Weston, and D. Kiela, "Adversarial NLI: A new benchmark for natural language understanding," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 4885–4901.

[30] M. Laurer, W. van Atteveldt, A. Casas, and K. Welbers, "Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI," *Political Analysis*, vol. 32, no. 1, pp. 84–100, 2024.

[31] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of ACL*, 2024.

[32] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[33] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[34] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30*, 2017.

[35] I. Goodfellow et al., "Generative adversarial nets," in *Advances in Neural Information Processing Systems 27*, 2014.

[36] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems 33*, 2020.

[37] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 8440–8451.

[38] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.

[39] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020, doi: 10.5281/zenodo.1212303.

[40] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[41] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[42] H. Rashkin, E. Choi, J. Y. Jang, S. Volkova, and Y. Choi, "Truth of varying shades: Analyzing language in fake news and political fact-checking," in *Proc. EMNLP*, 2017, pp. 2931–2937.

[43] X. Zhou and R. Zafarani, "A survey of fake news: Fundamental theories, detection methods, and opportunities," *ACM Computing Surveys*, vol. 53, no. 5, 2020.

[44] K. Shu, D. Mahudeswaran, S. Wang, D. Lee, and H. Liu, "FakeNewsNet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media," *Big Data*, vol. 8, no. 3, pp. 171–188, 2020.

[45] K. Popat, S. Mukherjee, A. Yates, and G. Weikum, "DeClarE: Debunking fake news and false claims using evidence-aware deep learning," in *Proc. EMNLP*, 2018.

[46] N. Kotonya and F. Toni, "Explainable automated fact-checking for public health claims," in *Proc. EMNLP*, 2020.

[47] P. Atanasova, J. G. Simonsen, C. Lioma, and I. Augenstein, "Generating fact checking explanations," in *Proc. 58th Annual Meeting of the ACL*, 2020.

[48] FastAPI documentation, https://fastapi.tiangolo.com (accessed Nov. 2026).

[49] React documentation, https://react.dev (accessed Nov. 2026).

[50] Google Fact Check Tools API documentation, https://developers.google.com/fact-check/tools/api (accessed Nov. 2026).